

final-melhorado-v2

June 16, 2026

```
[1]: '''  
    Bibliotecas usadas  
    '''  
  
    import os  
    from datetime import datetime  
    import re  
    import subprocess  
    import warnings  
    import numpy as np  
    import pandas as pd  
    from collections import Counter  
    from Bio import Entrez  
    from Bio import SeqIO  
    from Bio.Seq import Seq  
    from sklearn.model_selection import (  
        GroupShuffleSplit,  
        StratifiedGroupKFold,  
        RandomizedSearchCV,  
        GroupKFold  
    )  
    from sklearn.metrics import (  
        classification_report,  
        roc_auc_score,  
        average_precision_score,  
        make_scorer,  
        matthews_corrcoef,  
        f1_score,  
        precision_score,  
        recall_score,  
        balanced_accuracy_score,  
        accuracy_score,  
        confusion_matrix,  
        ConfusionMatrixDisplay,  
        roc_curve,  
        auc  
    )
```

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance

warnings.filterwarnings("ignore")

print("Bibliotecas importadas")

```

Bibliotecas importadas

```

[2]: '''
    obtenção dos datasets
    '''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAF"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# =====
# QUERIES
# =====

POSITIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum_
    ↳ melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis_
    ↳ peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA_
    ↳ OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N GENE"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"_
    ↳ OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" )
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR_
    ↳ "transcription factor" OR DNA-binding OR microtom)

```

```

    NOT ("pathogenesis-related protein" OR osmotin OR PR1 OR PR2 OR PR3 OR PR4
    ↪OR PR5 OR synthase OR metabolic OR precursor)
    NOT txid10239[Organism:exp]
'''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum
    ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
    AND 100:3000[Sequence Length]
    NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA
    ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N"
    OR "virus resistance" OR "antiviral" OR "Tobacco mosaic virus" OR "Potato
    ↪virus"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"
    ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RNA silencing" OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR
    ↪"eIF4E" )
    NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR
    ↪DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

ARABIDOPSIS_QUERY = '''
    "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

PHYSALIS_QUERY = '''
    "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
    AND 1500:30000[Sequence Length]
'''

# =====
# DOWNLOAD NCBI
# =====

```

```

def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n=====")
    print("DOWNLOAD NCBI: ", output_fasta)
    print("=====")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n=====")
    print("DOWNLOAD TRANSCRIPTOMA: ", output_fasta)
    print("=====")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

```

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas

```

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"

print("\n===== FINALIZADO CONFIGURAÇÃO DE PARÂMETROS =====\n")

```

===== FINALIZADO CONFIGURAÇÃO DE PARÂMETROS =====

```

[3]: inicio = datetime.now()
print("\n\nIniciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY, POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

Iniciando download da classe positiva: 16:26:17

```

=====
DOWNLOAD NCBI: pipelineAF/positive.fasta
=====
Proteínas encontradas: 8914
Arquivo salvo: pipelineAF/positive.fasta

```

Finalizado em: 16:27:46

Tempo decorrido: 0:01:29.540645

```

[4]: inicio = datetime.now()
print("\n\nIniciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY, NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====")

```

Iniciando download da classe negativa: 16:27:46

```

=====
DOWNLOAD NCBI: pipelineAF/negative.fasta
=====

```

Proteínas encontradas: 35000
Arquivo salvo: pipelineAF/negative.fasta

Finalizado em: 16:29:24

Tempo decorrido: 0:01:37.426665

Arquivos salvos na pasta: pipelineAF
===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====

```
[5]: #Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
↳fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")

[6]: print("\n\nIniciando download de dados de transcriptmas")
      inicio = datetime.now()
      print("\n\nIniciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
      baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
      fim = datetime.now()
      print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Iniciando download de dados de transcriptmas

Iniciando download ARABIDOPSIS: 16:29:24

=====

DOWNLOAD TRANSCRIPTOMA: pipelineAF/arabidopsis_transcriptoma.fasta

=====

Transcritos encontrados: 49951

Arquivo salvo: pipelineAF/arabidopsis_transcriptoma.fasta

Finalizado em: 16:30:09

Tempo decorrido: 0:00:44.844875

```
[7]: inicio = datetime.now()
      print("\n\nIniciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
      baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
```

```
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Iniciando download PHYSALIS: 16:30:09

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAF/physalis_transcriptoma.fasta
=====
Transcritos encontrados: 21702
Arquivo salvo: pipelineAF/physalis_transcriptoma.fasta
```

Finalizado em: 16:31:20

Tempo decorrido: 0:01:11.268131

```
[8]: inicio = datetime.now()
print("\n\nIniciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Iniciando download MELONGENA: 16:31:20

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAF/melongena_transcriptoma.fasta
=====
Transcritos encontrados: 8555
Arquivo salvo: pipelineAF/melongena_transcriptoma.fasta
```

Finalizado em: 16:31:57

Tempo decorrido: 0:00:37.571856

```
[9]: inicio = datetime.now()
print("\n\nIniciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
print("\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("===== FIM COLETA DE DADOS APLICAÇÃO =====")
```

Iniciando download TUBEROSUM: 16:31:57

```
=====
DOWNLOAD TRANSCRIPTOMA: pipelineAF/tuberosum_transcriptoma.fasta
=====
Transcritos encontrados: 20832
Arquivo salvo: pipelineAF/tuberosum_transcriptoma.fasta
```

Finalizado em: 16:32:42

Tempo decorrido: 0:00:44.502241

```
Arquivos salvos na pasta: pipelineAF
===== FIM COLETA DE DADOS APLICAÇÃO =====
```

```
[10]: '''
Tratamento dos dados
    - Criar Dataframes com os conjuntos de dados de treino e validação
    - Executar CD-Hit para eliminar sequencias muito semelhantes
    - Extração de atributos (features)

'''

# =====
# AAC
# =====
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de
↳aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o
↳modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # =====
```



```

# NLR (CNL/TNL)
# =====
"P_LOOP": r"[AGST]{3,5}GK[TS]",
"KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
"GLPL": r"G[LVI][PAST][LIVAF]",
"MHD": r"[MVIL][HN][DE][VILAST]",
"RNBS_A": r"F.DL.{0,3}AW",
"RNBS_B": r"G[SA]{2,6}T",
"RNBS_C": r"Y.[VIL]{1,3}[LS]",
"RNBS_D": r"C.{2,8}P",

# =====
# Protein Kinase (KIN/RLK)
# =====
"HRD": r"HRD[LIVMFY]",
"DFG": r"DFG",

# =====
# LRR (RLP/RLK)
# =====
"LRR1": r"L..L..L..N",
"LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"
}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# =====
# DIPEPTÍDEOS
# =====
'''
encontrar pares de aminoácidos consecutivos em uma sequência protéica □
↳ sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''

```

```

DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motivos.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes

def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):

```

```

    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]

# =====
# FEATURES - atributos
# =====
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos -
    ↪ frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera
    ↪ a ordem dos aminoácidos para buscar os mais frequentes em proteínas de
    ↪ resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glp1"])
    features.append(distancias["dist_glp1_mhd"])

    features.append(
        distancias["dist_ploop_kinase2"]/protein_length
    )

```

```

        if distancias["dist_ploop_kinase2"] != -1 else -1
    )
    features.append(
        distancias["dist_kinase2_glpl"]/protein_length
        if distancias["dist_kinase2_glpl"] != -1 else -1
    )
    features.append(
        distancias["dist_glpl_mhd"]/protein_length
        if distancias["dist_glpl_mhd"] != -1 else -1
    )

    features.append(int(posicoes["P_LOOP"] is not None))
    features.append(int(posicoes["KINASE2"] is not None))
    features.append(int(posicoes["GLPL"] is not None))
    features.append(int(posicoes["MHD"] is not None))

    encontrados = {}
    for nome, regex in motifs.items():
        encontrados[nome] = int( re.search(regex, seq) is not None)

    # features.append(encontrados[nome])

    num_nlr = (
        encontrados["P_LOOP"] +
        encontrados["KINASE2"] +
        encontrados["GLPL"] +
        encontrados["MHD"]
    )

    # num_kinase = (
    #     encontrados["HRD"] +
    #     encontrados["DFG"]
    # )

    # num_lrr = (
    #     encontrados["LRR1"] +
    #     encontrados["LRR2"]
    # )

    features.extend([
        num_nlr,
        # num_kinase,
        # num_lrr
    ])

    return features

```

```

# =====
# DATASET
# =====
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS"
]

# "NUM_KINASE_MOTIFS",
# "NUM_LRR_MOTIFS"

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue

```

```

        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

    df = pd.DataFrame(
        data,
        columns=columns + ["target", "id"]
    )
    print("\nTotal de Features/Atributos:", len(columns)) # quantidade
    print("\nFeatures/atributos:")
    print(columns)
    return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id

```

```

def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)
                clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\nIniciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nIniciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)
# =====
# Montar os DATASETS
# =====

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"

df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# =====
# CLUSTERS

```

```

# =====

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# =====
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# =====

print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |  

↳ Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os  

↳ negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),  

↳ random_state=42)
    df = pd.concat([df_positivos, df_negativos_balanceados]).  

↳ reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")

```



```

else:
    print("O dataset já está balanceado ou possui mais positivos que negativos,
    ↳ (incomum). Nenhuma ação foi tomada.")

print(f"Contagem final    -> Positivos: {len(df[df['target'] == 1])} | Negativos:
    ↳ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)

fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")

```

Iniciando Tratamento dos dados: 16:32:42

Iniciar a execução do CD-HIT

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAF/positive.fasta -o
         pipelineAF/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Mon Jun 15 16:32:42 2026

```

=====
                                Output
-----

```

```

total seq: 8914
longest and shortest : 3540 and 51
Total letters: 7801441
Sequences have been sorted

```

Approximated minimal memory consumption:

```

Sequence      : 8M
Buffer        : 4 X 11M = 45M
Table         : 2 X 65M = 130M
Miscellaneous  : 0M
Total         : 185M

```

Table limit with the given memory limit:

Max number of representatives: 4000000

Max number of word counting entries: 3726820522

```
# comparing sequences from      0  to      1485
.----- new table with      236 representatives
# comparing sequences from      1485  to      2723
99.9%----- new table with      192 representatives
# comparing sequences from      2723  to      3754
-----
      99 remaining sequences to the next cycle
----- new table with      164 representatives
# comparing sequences from      3655  to      4531
99.9%----- new table with      123 representatives
# comparing sequences from      4531  to      5261
99.7%----- new table with      63 representatives
# comparing sequences from      5261  to      5869
...----- new table with      88 representatives
# comparing sequences from      5869  to      6376
-----
     102 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      6274  to      6714
-----
      77 remaining sequences to the next cycle
----- new table with      116 representatives
# comparing sequences from      6637  to      7016
-----
      55 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      6961  to      7286
-----
      58 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      7228  to      7509
...----- new table with      76 representatives
# comparing sequences from      7509  to      7743
...----- new table with      66 representatives
# comparing sequences from      7743  to      7938
...----- new table with      48 representatives
# comparing sequences from      7938  to      8914
...----- new table with      247 representatives
```

8914 finished 1719 clusters

Approximated maximum memory consumption: 190M

writing new database

writing clustering information

program completed !

Total CPU time 3.61

CD-HIT concluído.

Executando CD-HIT...

=====

Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAF/negative.fasta -o
pipelineAF/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Mon Jun 15 16:32:43 2026

=====

Output

total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4204131
Sequences have been sorted

Approximated minimal memory consumption:

Sequence : 5M
Buffer : 4 X 11M = 44M
Table : 2 X 65M = 131M
Miscellaneous : 0M
Total : 181M

Table limit with the given memory limit:

Max number of representatives: 4000000
Max number of word counting entries: 3727298905

comparing sequences from 0 to 1666
----- new table with 1374 representatives
comparing sequences from 1666 to 3054
----- 497 remaining sequences to the next cycle
----- new table with 736 representatives
comparing sequences from 2557 to 3797
----- 784 remaining sequences to the next cycle
----- new table with 251 representatives
comparing sequences from 3013 to 4177
----- 784 remaining sequences to the next cycle
----- new table with 293 representatives
comparing sequences from 3393 to 4494
----- 723 remaining sequences to the next cycle
----- new table with 314 representatives
comparing sequences from 3771 to 4809
----- 715 remaining sequences to the next cycle
----- new table with 266 representatives
comparing sequences from 4094 to 5078
----- 642 remaining sequences to the next cycle
----- new table with 246 representatives
comparing sequences from 4436 to 5363
----- 628 remaining sequences to the next cycle
----- new table with 255 representatives
comparing sequences from 4735 to 5612

```

----- 549 remaining sequences to the next cycle
----- new table with      216 representatives
# comparing sequences from      5063 to      5885
----- 543 remaining sequences to the next cycle
----- new table with      216 representatives
# comparing sequences from      5342 to      6118
----- 522 remaining sequences to the next cycle
----- new table with      186 representatives
# comparing sequences from      5596 to      6329
----- 489 remaining sequences to the next cycle
----- new table with      170 representatives
# comparing sequences from      5840 to      6533
----- 462 remaining sequences to the next cycle
----- new table with      183 representatives
# comparing sequences from      6071 to      6725
----- 436 remaining sequences to the next cycle
----- new table with      167 representatives
# comparing sequences from      6289 to      6907
----- 414 remaining sequences to the next cycle
----- new table with      131 representatives
# comparing sequences from      6493 to      7077
----- 369 remaining sequences to the next cycle
----- new table with      146 representatives
# comparing sequences from      6708 to      7256
----- 327 remaining sequences to the next cycle
----- new table with      146 representatives
# comparing sequences from      6929 to      7440
----- 302 remaining sequences to the next cycle
----- new table with      142 representatives
# comparing sequences from      7138 to      7614
----- 269 remaining sequences to the next cycle
----- new table with      142 representatives
# comparing sequences from      7345 to      7787
----- 249 remaining sequences to the next cycle
----- new table with      125 representatives
# comparing sequences from      7538 to      7948
----- 246 remaining sequences to the next cycle
----- new table with      107 representatives
# comparing sequences from      7702 to      8084
----- 211 remaining sequences to the next cycle
----- new table with      105 representatives
# comparing sequences from      7873 to      8227
----- 211 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8016 to      8346
----- 176 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8170 to      8474

```

```

----- 171 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8303 to 8585
----- 130 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8455 to 8712
----- 10 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8702 to 8918
----- 75 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8843 to 9035
----- 48 remaining sequences to the next cycle
----- new table with 100 representatives
# comparing sequences from 8987 to 9155
...----- new table with 70 representatives
# comparing sequences from 9155 to 9999
...----- new table with 405 representatives

```

9999 finished 7092 clusters

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

Total CPU time 1.52
CD-HIT concluído.

Total de Features/Atributos: 445

Features/atributos:

```

['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',

```

'DIEP_FC', 'DIEP_FD', 'DIEP_FE', 'DIEP_FF', 'DIEP_FG', 'DIEP_FH',
 'DIEP_FI', 'DIEP_FK', 'DIEP_FL', 'DIEP_FM', 'DIEP_FN', 'DIEP_FP',
 'DIEP_FQ', 'DIEP_FR', 'DIEP_FS', 'DIEP_FT', 'DIEP_FV', 'DIEP_FW',
 'DIEP_FY', 'DIEP_GA', 'DIEP_GC', 'DIEP_GD', 'DIEP_GE', 'DIEP_GF',
 'DIEP_GG', 'DIEP_GH', 'DIEP_GI', 'DIEP_GK', 'DIEP_GL', 'DIEP_GM',
 'DIEP_GN', 'DIEP_GP', 'DIEP_GQ', 'DIEP_GR', 'DIEP_GS', 'DIEP_GT',
 'DIEP_GV', 'DIEP_GW', 'DIEP_GY', 'DIEP_HA', 'DIEP_HC', 'DIEP_HD',
 'DIEP_HE', 'DIEP_HF', 'DIEP_HG', 'DIEP_HH', 'DIEP_HI', 'DIEP_HK',
 'DIEP_HL', 'DIEP_HM', 'DIEP_HN', 'DIEP_HP', 'DIEP_HQ', 'DIEP_HR',
 'DIEP_HS', 'DIEP_HT', 'DIEP_HV', 'DIEP_HW', 'DIEP_HY', 'DIEP_IA',
 'DIEP_IC', 'DIEP_ID', 'DIEP_IE', 'DIEP_IF', 'DIEP_IG', 'DIEP_IH',
 'DIEP_II', 'DIEP_IK', 'DIEP_IL', 'DIEP_IM', 'DIEP_IN', 'DIEP_IP',
 'DIEP_IQ', 'DIEP_IR', 'DIEP_IS', 'DIEP_IT', 'DIEP_IV', 'DIEP_IW',
 'DIEP_IY', 'DIEP_KA', 'DIEP_KC', 'DIEP_KD', 'DIEP_KE', 'DIEP_KF',
 'DIEP_KG', 'DIEP_KH', 'DIEP_KI', 'DIEP_KK', 'DIEP_KL', 'DIEP_KM',
 'DIEP_KN', 'DIEP_KP', 'DIEP_KQ', 'DIEP_KR', 'DIEP_KS', 'DIEP_KT',
 'DIEP_KV', 'DIEP_KW', 'DIEP_KY', 'DIEP_LA', 'DIEP_LC', 'DIEP_LD',
 'DIEP_LE', 'DIEP_LF', 'DIEP_LG', 'DIEP_LH', 'DIEP_LI', 'DIEP_LK',
 'DIEP_LL', 'DIEP_LM', 'DIEP_LN', 'DIEP_LP', 'DIEP_LQ', 'DIEP_LR',
 'DIEP_LS', 'DIEP_LT', 'DIEP_LV', 'DIEP_LW', 'DIEP_LY', 'DIEP_MA',
 'DIEP_MC', 'DIEP_MD', 'DIEP_ME', 'DIEP_MF', 'DIEP_MG', 'DIEP_MH',
 'DIEP_MI', 'DIEP_MK', 'DIEP_ML', 'DIEP_MM', 'DIEP_MN', 'DIEP_MP',
 'DIEP_MQ', 'DIEP_MR', 'DIEP_MS', 'DIEP_MT', 'DIEP_MV', 'DIEP_MW',
 'DIEP_MY', 'DIEP_NA', 'DIEP_NC', 'DIEP_ND', 'DIEP_NE', 'DIEP_NF',
 'DIEP_NG', 'DIEP_NH', 'DIEP_NI', 'DIEP_NK', 'DIEP_NL', 'DIEP_NM',
 'DIEP_NN', 'DIEP_NP', 'DIEP_NQ', 'DIEP_NR', 'DIEP_NS', 'DIEP_NT',
 'DIEP_NV', 'DIEP_NW', 'DIEP_NY', 'DIEP_PA', 'DIEP_PC', 'DIEP_PD',
 'DIEP_PE', 'DIEP_PF', 'DIEP_PG', 'DIEP_PH', 'DIEP_PI', 'DIEP_PK',
 'DIEP_PL', 'DIEP_PM', 'DIEP_PN', 'DIEP_PP', 'DIEP_PQ', 'DIEP_PR',
 'DIEP_PS', 'DIEP_PT', 'DIEP_PV', 'DIEP_PW', 'DIEP_PY', 'DIEP_QA',
 'DIEP_QC', 'DIEP_QD', 'DIEP_QE', 'DIEP_QF', 'DIEP_QG', 'DIEP_QH',
 'DIEP_QI', 'DIEP_QK', 'DIEP_QL', 'DIEP_QM', 'DIEP_QN', 'DIEP_QP',
 'DIEP_QQ', 'DIEP_QR', 'DIEP_QS', 'DIEP_QT', 'DIEP_QV', 'DIEP_QW',
 'DIEP_QY', 'DIEP_RA', 'DIEP_RC', 'DIEP_RD', 'DIEP_RE', 'DIEP_RF',
 'DIEP_RG', 'DIEP_RH', 'DIEP_RI', 'DIEP_RK', 'DIEP_RL', 'DIEP_RM',
 'DIEP_RN', 'DIEP_RP', 'DIEP_RQ', 'DIEP_RR', 'DIEP_RS', 'DIEP_RT',
 'DIEP_RV', 'DIEP_RW', 'DIEP_RY', 'DIEP_SA', 'DIEP_SC', 'DIEP_SD',
 'DIEP_SE', 'DIEP_SF', 'DIEP_SG', 'DIEP_SH', 'DIEP_SI', 'DIEP_SK',
 'DIEP_SL', 'DIEP_SM', 'DIEP_SN', 'DIEP_SP', 'DIEP_SQ', 'DIEP_SR',
 'DIEP_SS', 'DIEP_ST', 'DIEP_SV', 'DIEP_SW', 'DIEP_SY', 'DIEP_TA',
 'DIEP_TC', 'DIEP_TD', 'DIEP_TE', 'DIEP_TF', 'DIEP_TG', 'DIEP_TH',
 'DIEP_TI', 'DIEP_TK', 'DIEP_TL', 'DIEP_TM', 'DIEP_TN', 'DIEP_TP',
 'DIEP_TQ', 'DIEP_TR', 'DIEP_TS', 'DIEP_TT', 'DIEP_TV', 'DIEP_TW',
 'DIEP_TY', 'DIEP_VA', 'DIEP_VC', 'DIEP_VD', 'DIEP_VE', 'DIEP_VF',
 'DIEP_VG', 'DIEP_VH', 'DIEP_VI', 'DIEP_VK', 'DIEP_VL', 'DIEP_VM',
 'DIEP_VN', 'DIEP_VP', 'DIEP_VQ', 'DIEP_VR', 'DIEP_VS', 'DIEP_VT',
 'DIEP_VV', 'DIEP_VW', 'DIEP_VY', 'DIEP_WA', 'DIEP_WC', 'DIEP_WD',
 'DIEP_WE', 'DIEP_WF', 'DIEP_WG', 'DIEP_WH', 'DIEP_WI', 'DIEP_WK',

```
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS',
'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']
```

Clusters ausentes:
164

Dataset original pós CD-HIT: (8647, 448)

--- Executando Balanceamento de Classes ---

Contagem inicial -> Positivos (R-genes): 1700 | Negativos (Background): 6947

Subamostragem aplicada com sucesso nos Negativos.

Contagem final -> Positivos: 1700 | Negativos: 1700

Dataset Final Pronto: (3400, 448)

Finalizado em: 16:32:46

Tempo decorrido: 0:00:03.586248

Fim tratamento dos dados...

```
[11]: '''
      k-Fold-Cross-Validation
      '''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=5, #10
        shuffle=True,
        random_state=42
    )
    mcc_scorer = make_scorer(matthews_corrcoef)
    scoring_dict = {
        'recall': 'recall',
        'roc_auc': 'roc_auc',
        'mcc': mcc_scorer,
```

```

        'accuracy': 'accuracy',
        'precision': 'precision',
        'f1': 'f1'
    }
    scores = cross_validate(
        model,
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=scoring_dict,
        n_jobs=-1
    )
    print(f"Finalizada a validação cruzada para: {nome}")
    return scores

```

```

[12]: '''
Criação dos modelos/ Treinamento
'''
# para melhor separar os conjuntos de treino e teste considerando os
# ↳ agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """

```



```

    Calcula os SHAP values para modelos de árvore e salva os gráficos
    ↪ explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
    ↪")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # 0 Random Forest do sklearn retorna uma lista contendo [valores_classe_0,
    ↪valores_classe_1]
    # 0 XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
        ↪alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência
    ↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20, # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",
    ↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))

```

```

shap.summary_plot(
    shap_values_pos,
    X_test,
    feature_names=feature_names,
    plot_type="bar",
    max_display=20,
    show=False
)
# plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",
↳dpi=300)
plt.close()

print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")

def calcular_shap_svm(nome, model, feature_names, X_train, X_test, y_test,
↳output_dir):
    # =====
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # =====
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

    # Calcula a permutação no X_test (pode usar X_train se quiser focar no
↳ajuste ao treino)
    result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,
↳random_state=42, n_jobs=-1)

    # Monta o DataFrame com os resultados
    importancia_svm = pd.DataFrame({
        'feature': columns,
        'importance_mean': result_svm.importances_mean
    }).sort_values('importance_mean', ascending=False)

    print("\nTop 20 features mais importantes para o SVM:")
    print(importancia_svm.head(20))

    # Gera o gráfico de barras igual ao do XGBoost
    top_svm = importancia_svm.head(20)
    plt.figure(figsize=(10,8))
    plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],
↳color='seagreen')
    plt.xlabel("Queda no Desempenho (Média)")
    plt.ylabel("Feature / Atributo")
    # plt.title("Top 20 Features - SVM Permutation Importance")
    plt.tight_layout()

```

```

plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",
↳bbox_inches="tight", dpi=300)
plt.close()

idx_back = np.random.choice(
    len(X_train),
    min(100, len(X_train)),
    replace = False
)
back = X_train[idx_back]

idx_test = np.random.choice(
    len(X_test),
    min(200, len(X_test)),
    replace = False
)
x_test_sample = X_test[idx_test]

print('\nCriando explainer shap...\n')
explainer = shap.Explainer(model.predict, back)
shap_values = explainer(x_test_sample, max_evals=1000)
print("\nShap SHAP:", shap_values.values.shape)

plt.figure(figsize=(12, 8))
shap.summary_plot(
    shap_values.values,
    x_test_sample,
    feature_names=feature_names,
    # plot_type="bar",
    max_display=20,
    show=False
)
# plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",
↳bbox_inches="tight", dpi=300)
plt.close()

print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/
↳svm_importancia_permutacao.png'!")

# =====
# TREINAMENTO - treina os modelos
# =====

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n=====")

```

```

print(nome)
print("=====")
model.fit(X_train, y_train)

probs = model.predict_proba(X_test)[: ,1]
preds = (probs >= 0.4).astype(int)
print(classification_report(y_test, preds))
roc = roc_auc_score(y_test, probs)
pr = average_precision_score(y_test, probs)
mcc = matthews_corrcoef(y_test, preds)
f1 = f1_score(y_test, preds)
precision = precision_score(y_test, preds)
recall = recall_score(y_test, preds)
bal_acc = balanced_accuracy_score(y_test, preds)
acc = accuracy_score(y_test, preds)

print("ROC-AUC          :", roc)
print("PR-AUC          :", pr)
print("MCC             :", mcc)
print("F1              :", f1)
print("Precision       :", precision)
print("Recall          :", recall)
print("Balanced Accuracy:", bal_acc)
print("Standard Accuracy:", acc)

# =====
# MATRIZ DE CONFUSÃO
# =====
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(5,5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativo", "Positivo"]
)
disp.plot(cmap="Blues")
# plt.title(f"Matriz de Confusão - {nome}")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
    bbox_inches="tight"
)
plt.close()

# =====
# CURVA ROC
# =====
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))

```

```

plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.4f}"
)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# plt.title(f"Curva ROC - {nome}")
plt.legend(loc="lower right")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_roc.png",
    bbox_inches="tight"
)
plt.close()

return {
    "Modelo": nome,
    "ROC_AUC": roc,
    "F1": f1,
    "Precision": precision,
    "Recall": recall,
    "Balanced_Accuracy": bal_acc,
    "Accuracy": acc,
    "MCC": mcc
}, model

# =====
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# =====
inicio_treinamento = datetime.now()
print("\n\nIniciando Treinamento dos modelos: ", inicio_treinamento.
    ↳strftime("%H:%M:%S"))

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# MODELOS

```

```

# =====

rf = RandomForestClassifier(
    n_estimators=1000,
    # max_depth = 12,
    max_depth = None,
    min_samples_split=5,
    max_features=0.2,
    random_state=42,
    class_weight="balanced",
    n_jobs = 1
)

xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=7,
    learning_rate=0.05,
    # early_stopping_rounds=50,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.0
)

svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)

scores = []
groups_train = train_df["cluster"]

# msc_scorer = make_scorer(matthews_corrcoef)

scoring = {
    'mcc': make_scorer(matthews_corrcoef),
    'recall': make_scorer(recall_score),
    'precision': make_scorer(precision_score),
    'f1': make_scorer(f1_score),

```

```
'roc_auc': 'roc_auc',
}
```

Iniciando Treinamento dos modelos: 16:32:46

	protein_length	AAC_A	AAC_C	AAC_D	AAC_E	AAC_F \
0	231	0.090909	0.012987	0.069264	0.077922	0.051948
1	704	0.049716	0.007102	0.080966	0.117898	0.041193
2	971	0.045314	0.017508	0.054583	0.071061	0.036045
3	323	0.046440	0.027864	0.052632	0.080495	0.046440
4	929	0.034446	0.034446	0.046286	0.069968	0.053821
5	709	0.053597	0.014104	0.049365	0.071932	0.043724
6	999	0.053053	0.018018	0.070070	0.050050	0.036036
7	1071	0.056956	0.020542	0.059757	0.038282	0.043884
8	1050	0.043810	0.022857	0.053333	0.080952	0.041905
9	904	0.042035	0.022124	0.032080	0.055310	0.056416
10	1152	0.065972	0.015625	0.041667	0.046875	0.030382
11	949	0.033720	0.034773	0.052687	0.067439	0.041096
12	272	0.055147	0.033088	0.040441	0.084559	0.040441
13	428	0.049065	0.023364	0.070093	0.070093	0.051402
14	253	0.059289	0.019763	0.071146	0.051383	0.039526
15	886	0.038375	0.024831	0.060948	0.055305	0.050790
16	910	0.052747	0.017582	0.060440	0.065934	0.052747
17	879	0.035267	0.018203	0.052332	0.073948	0.046644
18	239	0.041841	0.020921	0.083682	0.050209	0.046025
19	126	0.055556	0.031746	0.055556	0.055556	0.039683
20	104	0.028846	0.038462	0.038462	0.067308	0.028846
21	105	0.057143	0.019048	0.076190	0.019048	0.057143
22	716	0.044693	0.023743	0.061453	0.067039	0.046089
23	196	0.061224	0.010204	0.040816	0.091837	0.035714
24	122	0.040984	0.008197	0.081967	0.073770	0.040984
25	1289	0.044220	0.017843	0.061288	0.074476	0.046548
26	846	0.034279	0.017730	0.063830	0.069740	0.047281
27	883	0.055493	0.019253	0.057758	0.070215	0.048698
28	906	0.052980	0.020971	0.055188	0.073951	0.049669
29	343	0.084548	0.011662	0.061224	0.064140	0.037901

	AAC_G	AAC_H	AAC_I	AAC_K	...	dist_ploop_kinase2_norm \
0	0.077922	0.038961	0.043290	0.082251	...	-1.000000
1	0.048295	0.009943	0.065341	0.117898	...	-1.000000
2	0.055613	0.024717	0.057673	0.061792	...	0.094748
3	0.065015	0.024768	0.061920	0.052632	...	-1.000000
4	0.051668	0.027987	0.071044	0.059203	...	-1.000000
5	0.063470	0.021157	0.063470	0.077574	...	-1.000000
6	0.069069	0.021021	0.043043	0.077077	...	-1.000000
7	0.090570	0.023343	0.040149	0.059757	...	-1.000000

8	0.048571	0.024762	0.065714	0.058095	...	-0.023810
9	0.059735	0.028761	0.065265	0.074115	...	-1.000000
10	0.110243	0.026910	0.039062	0.046007	...	-1.000000
11	0.049526	0.025290	0.060063	0.052687	...	0.089568
12	0.044118	0.029412	0.047794	0.066176	...	-1.000000
13	0.053738	0.011682	0.072430	0.077103	...	-1.000000
14	0.063241	0.027668	0.075099	0.102767	...	0.280632
15	0.053047	0.027088	0.067720	0.082393	...	0.106095
16	0.040659	0.021978	0.071429	0.086813	...	0.100000
17	0.039818	0.036405	0.069397	0.059158	...	0.102389
18	0.033473	0.025105	0.092050	0.079498	...	-1.000000
19	0.055556	0.039683	0.079365	0.095238	...	-1.000000
20	0.067308	0.019231	0.067308	0.076923	...	-1.000000
21	0.057143	0.019048	0.066667	0.076190	...	-1.000000
22	0.051676	0.018156	0.064246	0.071229	...	-1.000000
23	0.035714	0.020408	0.071429	0.076531	...	-1.000000
24	0.032787	0.016393	0.049180	0.073770	...	-1.000000
25	0.038014	0.038014	0.071373	0.074476	...	0.070597
26	0.033097	0.027187	0.075650	0.065012	...	0.106383
27	0.049830	0.022650	0.056625	0.092865	...	0.098528
28	0.045254	0.029801	0.070640	0.071744	...	0.097130
29	0.075802	0.032070	0.075802	0.067055	...	-1.000000

	dist_kinase2_glp1_norm	dist_glp1_mhd_norm	has_ploop	has_kinase2	\
0	-1.000000	1	0	0	
1	-1.000000	0	0	1	
2	0.108136	1	1	1	
3	-1.000000	0	0	0	
4	-1.000000	1	0	1	
5	0.128350	0	0	1	
6	0.288288	1	0	1	
7	0.202614	0	0	1	
8	0.096190	1	1	1	
9	-1.000000	0	0	1	
10	-1.000000	0	0	1	
11	0.295047	1	1	1	
12	0.312500	0	0	1	
13	0.235981	1	0	1	
14	-1.000000	1	1	1	
15	0.093679	1	1	1	
16	-0.346154	1	1	1	
17	0.145620	1	1	1	
18	-1.000000	1	0	0	
19	-1.000000	0	0	0	
20	-1.000000	0	0	1	
21	-1.000000	1	0	0	
22	0.139665	1	0	1	
23	-1.000000	0	1	0	

24	-1.000000	0	0	0
25	-0.452289	1	1	1
26	0.102837	1	1	1
27	0.412231	1	1	1
28	0.143488	1	1	1
29	-1.000000	0	0	0

	has_glp1	has_mhd	target	id	cluster
0	0	1	1	NP_001234459.3	POS_1502
1	0	1	1	NP_001308492.1	POS_1005
2	1	4	1	CAQ5402917.1	POS_548
3	0	0	1	CAQ5403257.1	POS_1404
4	0	2	1	CAQ5404663.1	POS_614
5	1	2	1	CAQ5406153.1	POS_1000
6	1	3	1	CAQ5404100.1	POS_504
7	1	2	1	CAQ5404098.1	POS_427
8	1	4	1	CAQ5403806.1	POS_449
9	0	1	1	CAQ5409432.1	POS_681
10	0	1	1	CAQ5408022.1	POS_356
11	1	4	1	CAQ5407207.1	POS_585
12	1	2	1	CAQ5411306.1	POS_1456
13	1	3	1	CAQ5411102.1	POS_1278
14	0	3	1	CAQ5410995.1	POS_1472
15	1	4	1	CAQ5413609.1	POS_723
16	1	4	1	CAQ5413612.1	POS_663
17	1	4	1	CAQ5410853.1	POS_740
18	0	1	1	CAQ5410852.1	POS_1491
19	0	0	1	CAQ5413789.1	POS_1696
20	0	1	1	CAQ5410659.1	POS_1709
21	0	1	1	CAQ5410656.1	POS_1708
22	1	3	1	CAQ5410654.1	POS_994
23	1	2	1	CAQ5410652.1	POS_1573
24	1	1	1	CAQ5410649.1	POS_1699
25	1	4	1	CAQ5410600.1	POS_211
26	1	4	1	CAQ5413906.1	POS_810
27	1	4	1	CAQ5410544.1	POS_729
28	1	4	1	CAQ5413907.1	POS_675
29	0	0	1	CAQ5413931.1	POS_1378

[30 rows x 448 columns]

```
[13]: inicio_otimizacao = datetime.now()
print("\n\nIniciando Otimização dos modelos: ", inicio_otimizacao.strftime("%H:
↪%M:%S"))

cv_t = GroupKFold(n_splits=5)
```

```

param_dist_xgb = {
    'n_estimators': [500, 800, 1200, 1500],
    'max_depth': [3, 4, 5, 6, 7, 10],
    'learning_rate': [0.01, 0.03, 0.05, 0.1],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9],
    'min_child_weight': [1, 3, 5],
    'reg_alpha': [0, 0.1, 0.5, 0.7],
    'reg_lambda': [1, 2, 5]
}

# Otimização XGB
print('\n\nOtimizando XGBoost...')
search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist_xgb,
    n_iter=50,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_xgb.fit(
    X_train,
    y_train,
    groups=groups_train
)

print("\nMelhores parâmetros para o XGB:")
print(search_xgb.best_params_)

print("\nMelhor Recall:")
print(search_xgb.best_score_)

best_params = search_xgb.best_params_
xgb = search_xgb.best_estimator_

```

Iniciando Otimização dos modelos: 16:32:46

Otimizando XGBoost...

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Melhores parâmetros para o XGB:

```
{'subsample': 0.8, 'reg_lambda': 1, 'reg_alpha': 0.5, 'n_estimators': 1500,
'min_child_weight': 1, 'max_depth': 3, 'learning_rate': 0.05,
'colsample_bytree': 0.7}
```

Melhor Recall:

0.8924945736710903

```
[14]: # otimização RF

param_dist_rf = {
    'n_estimators': [500, 1000, 1200, 1500],
    'max_depth': [8, 12, 16, None],
    'min_samples_split': [2, 4, 5, 8],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 0, 3],
    'class_weight': ['balanced', 'balanced_subsample']
}

print('\n\nOtimizando RF...')
search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist_rf,
    n_iter=40,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_rf.fit(
    X_train,
    y_train,
    groups=groups_train
)

print("\nMelhores parâmetros para o RF:")
print(search_rf.best_params_)

print("\nMelhor Recall:")
print(search_rf.best_score_)

rf = search_rf.best_estimator_

rf.set_params(n_jobs=-1)
```

Otimizando RF...

Fitting 5 folds for each of 40 candidates, totalling 200 fits

Melhores parâmetros para o RF:

```
{ 'n_estimators': 1500, 'min_samples_split': 4, 'min_samples_leaf': 1,  
'max_features': 'sqrt', 'max_depth': 16, 'class_weight': 'balanced_subsample' }
```

Melhor Recall:

0.8616545159921097

```
[14]: RandomForestClassifier(class_weight='balanced_subsample', max_depth=16,  
                             min_samples_split=4, n_estimators=1500, n_jobs=-1,  
                             random_state=42)
```

```
[15]: print('\n\nOtimizando SVM...')  
param_dist_svm = {  
    'C': [0.5, 1, 2, 5, 10, 20, 50],  
    'gamma': [  
        'scale',  
        0.01,  
        0.005,  
        0.001,  
        0.0005  
    ],  
    'kernel': ['rbf']  
}  
search_svm = RandomizedSearchCV(  
    estimator=svm,  
    param_distributions=param_dist_svm,  
    n_iter=25,  
    scoring=scoring,  
    refit='f1',  
    cv=cv_t,  
    random_state=42,  
    verbose=2,  
    n_jobs=-1  
)  
  
search_svm.fit(  
    X_train,  
    y_train,  
    groups=groups_train  
)  
print("\nMelhores parâmetros para o SVM:")  
print(search_svm.best_params_)  
  
print("\nMelhor Recall:")
```

```

print(search_svm.best_score_)

svm = search_svm.best_estimator_

fim = datetime.now()
print("\n\nFinalizado otimização dos modelos em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_otimizacao)
# Fim otimização

```

Otimizando SVM...

Fitting 5 folds for each of 25 candidates, totalling 125 fits

Melhores parâmetros para o SVM:

```
{'kernel': 'rbf', 'gamma': 0.001, 'C': 2}
```

Melhor Recall:

0.88953740935566

Finalizado otimização dos modelos em: 16:48:06

Tempo decorrido: 0:15:20.702398

```

[16]: # xgb = search_xgb.best_estimator_

print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

```

```

resultados_cv = pd.DataFrame({
    'Modelo': ['RF', 'XGB', 'SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
    f"{OUTPUT_DIR}/metricas_cv.csv",
    index=False
)
datos = []

for score in scores_rf['test_roc_auc']:
    datos.append(['RF', score])
for score in scores_xgb['test_roc_auc']:
    datos.append(['XGB', score])
for score in scores_svm['test_roc_auc']:
    datos.append(['SVM', score])

```

```

grafico = pd.DataFrame(dados, columns=['Modelo', 'ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()

```

Iniciando validação cruzada
 Finalizada a validação cruzada para: RF
 Finalizada a validação cruzada para: XGB
 Finalizada a validação cruzada para: SVM

Cross-Validation:RF

test_recall: 0.7670 ± 0.0170
 test_roc_auc: 0.9380 ± 0.0065
 test_mcc: 0.7691 ± 0.0131
 test_accuracy: 0.8746 ± 0.0076
 test_precision: 0.9816 ± 0.0092
 test_f1: 0.8610 ± 0.0095

Cross-Validation XGB

test_recall: 0.8491 ± 0.0185
 test_roc_auc: 0.9411 ± 0.0070
 test_mcc: 0.7850 ± 0.0166
 test_accuracy: 0.8908 ± 0.0089
 test_precision: 0.9294 ± 0.0061
 test_f1: 0.8873 ± 0.0102

Cross-Validation SVM

test_recall: 0.8404 ± 0.0209
 test_roc_auc: 0.9459 ± 0.0096
 test_mcc: 0.8018 ± 0.0216
 test_accuracy: 0.8978 ± 0.0114
 test_precision: 0.9525 ± 0.0129
 test_f1: 0.8927 ± 0.0126

	Modelo	Accuracy	Recall	Precision	F1	ROC_AUC	MCC
0	RF	0.874632	0.767049	0.981621	0.861010	0.938043	0.769082
1	XGB	0.890809	0.849059	0.929357	0.887273	0.941087	0.785004
2	SVM	0.897794	0.840353	0.952514	0.892734	0.945878	0.801819

```

[17]: #####
# Treinamento
inicio = datetime.now()
print("\n\nIniciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns, OUTPUT_DIR )

inicio = datetime.now()
print("\n\nIniciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns, OUTPUT_DIR )

inicio = datetime.now()
print("\n\nIniciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)

```



```

results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model, columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")

```

Iniciando treinamento RF: 16:48:31

Random Forest

	precision	recall	f1-score	support
0	0.90	0.93	0.91	358
1	0.92	0.89	0.90	322
accuracy			0.91	680
macro avg	0.91	0.91	0.91	680
weighted avg	0.91	0.91	0.91	680

ROC-AUC : 0.9544224296471077
 PR-AUC : 0.9575617197127616
 MCC : 0.8172051864665247
 F1 : 0.9022082018927445
 Precision : 0.9166666666666666
 Recall : 0.8881987577639752
 Balanced Accuracy: 0.9077865297199764
 Standard Accuracy: 0.9088235294117647

Finalizado treinamento RF em: 16:48:35

Tempo decorrido: 0:00:03.434557

[SHAP] Inicializando análise de interpretabilidade para: Random Forest...
 [SHAP] Gráficos salvos com sucesso em 'pipelineAF/'!

Iniciando treinamento XGBoost: 16:49:23

```
=====
XGBoost
=====
```

	precision	recall	f1-score	support
0	0.90	0.93	0.91	358
1	0.92	0.89	0.90	322
accuracy			0.91	680
macro avg	0.91	0.91	0.91	680
weighted avg	0.91	0.91	0.91	680

ROC-AUC : 0.9580138103334606
PR-AUC : 0.9569489321363452
MCC : 0.8142955336607355
F1 : 0.9004739336492891
Precision : 0.9163987138263665
Recall : 0.8850931677018633
Balanced Accuracy: 0.9062337346889204
Standard Accuracy: 0.9073529411764706

Finalizado em XGBoost: 16:49:27

Tempo decorrido: 0:00:04.454943

[SHAP] Inicializando análise de interpretabilidade para: XGBoost...
[SHAP] Gráficos salvos com sucesso em 'pipelineAF/'!

Iniciando treinamento SVM: 16:49:28

```
=====
SVM
=====
```

	precision	recall	f1-score	support
0	0.92	0.89	0.90	358
1	0.88	0.91	0.89	322
accuracy			0.90	680
macro avg	0.90	0.90	0.90	680
weighted avg	0.90	0.90	0.90	680

ROC-AUC : 0.9522363718380236
PR-AUC : 0.9548255783478949
MCC : 0.7944209960287347
F1 : 0.8932926829268293
Precision : 0.8772455089820359
Recall : 0.9099378881987578
Balanced Accuracy: 0.8977063742669766
Standard Accuracy: 0.8970588235294118

Finalizado em SVM: 16:49:30

Tempo decorrido: 0:00:01.557960

[SVM] Calculando a importância por permutação dos atributos...

Top 20 features mais importantes para o SVM:

	feature	importance_mean
150	DIPEP_HL	0.008235
250	DIPEP_NL	0.007647
38	DIPEP_AV	0.006176
215	DIPEP_LR	0.006176
204	DIPEP_LE	0.005882
209	DIPEP_LK	0.005588
373	DIPEP_VP	0.005294
351	DIPEP_TM	0.005000
73	DIPEP_DP	0.004706
24	DIPEP_AE	0.004412
188	DIPEP_KI	0.004412
45	DIPEP_CF	0.004118
346	DIPEP_TG	0.003824
81	DIPEP_EA	0.003529
98	DIPEP_EV	0.003529
65	DIPEP_DF	0.003529
265	DIPEP_PF	0.003529
321	DIPEP_SA	0.003235
213	DIPEP_LP	0.003235
132	DIPEP_GN	0.003235

Criando explainer shap...

PermutationExplainer explainer: 9% | 18/200 [04:55<54:43, 18.04s/it]

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 28.8s

[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,

```

subsample=0.7; total time= 41.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 41.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 52.8s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 45.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 49.1s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 30.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 36.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.4min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 4.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time= 2.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 6.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time= 7.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 23.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 8.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 6.1s
[CV] END class_weight=balanced, max_depth=None, max_features=0,

```

```

min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time= 5.3s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 7.7s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 18.9s
[CV] END ...C=10, gamma=0.0005, kernel=rbf; total time= 15.9s
[CV] END ...C=5, gamma=scale, kernel=rbf; total time= 31.4s
[CV] END ...C=1, gamma=0.0005, kernel=rbf; total time= 21.5s
[CV] END ...C=0.5, gamma=scale, kernel=rbf; total time= 23.7s
[CV] END ...C=1, gamma=scale, kernel=rbf; total time= 24.5s
[CV] END ...C=50, gamma=0.005, kernel=rbf; total time= 37.0s
[CV] END ...C=50, gamma=0.01, kernel=rbf; total time= 7.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 57.5s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 51.6s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 36.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,

```

```

min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time= 35.7s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 36.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 44.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 25.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time= 38.4s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 35.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 57.2s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 43.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 29.2s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 24.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time= 21.1s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time= 21.6s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 22.8s
[CV] END ...C=2, gamma=0.001, kernel=rbf; total time= 20.1s
[CV] END ...C=20, gamma=0.0005, kernel=rbf; total time= 15.5s
[CV] END ...C=5, gamma=0.0005, kernel=rbf; total time= 16.0s
[CV] END ...C=1, gamma=0.0005, kernel=rbf; total time= 19.2s
[CV] END ...C=0.5, gamma=0.0005, kernel=rbf; total time= 22.0s
[CV] END ...C=1, gamma=scale, kernel=rbf; total time= 24.1s
[CV] END ...C=50, gamma=0.005, kernel=rbf; total time= 35.2s
[CV] END ...C=50, gamma=0.01, kernel=rbf; total time= 9.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 18.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,

```

```

min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 40.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 50.7s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 53.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 42.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 47.6s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time= 28.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 21.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 37.0s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 24.7s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time= 35.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 26.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 5.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time= 5.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time= 11.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time= 11.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time= 8.5s

```

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time= 8.4s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 10.1s

[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time= 5.3s

[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 7.9s

[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 9.3s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 6.5s

[CV] END ...C=20, gamma=0.01, kernel=rbf; total time= 40.7s

[CV] END ...C=1, gamma=0.001, kernel=rbf; total time= 17.5s

[CV] END ...C=50, gamma=0.001, kernel=rbf; total time= 15.4s

[CV] END ...C=5, gamma=0.005, kernel=rbf; total time= 36.3s

[CV] END ...C=0.5, gamma=0.01, kernel=rbf; total time= 36.1s

[CV] END ...C=10, gamma=0.001, kernel=rbf; total time= 13.1s

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 19.8s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 40.6s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 50.6s

[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 52.8s

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 40.4s

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.4min

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 57.5s

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 24.4s

[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time= 37.7s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 23.9s

[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 23.8s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time= 21.2s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time= 20.6s

[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 22.9s

[CV] END ...C=2, gamma=0.001, kernel=rbf; total time= 19.3s

[CV] END ...C=20, gamma=0.0005, kernel=rbf; total time= 16.5s

[CV] END ...C=2, gamma=0.005, kernel=rbf; total time= 36.3s

[CV] END ...C=0.5, gamma=0.0005, kernel=rbf; total time= 22.1s

[CV] END ...C=2, gamma=0.01, kernel=rbf; total time= 39.2s

[CV] END ...C=0.5, gamma=0.001, kernel=rbf; total time= 21.2s

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 58.0s

[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min

[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.1min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 36.8s

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time= 36.1s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 36.5s

[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 43.8s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 26.1s

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,

```

subsample=0.7; total time= 38.5s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 35.5s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 31.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 13.5s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 24.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time= 36.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 21.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 4.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time= 2.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 6.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time= 0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time= 7.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,

```

```

min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time= 7.6s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time= 21.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time= 21.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 23.6s
[CV] END ...C=10, gamma=0.01, kernel=rbf; total time= 40.3s
[CV] END ...C=1, gamma=0.001, kernel=rbf; total time= 22.0s
[CV] END ...C=50, gamma=0.001, kernel=rbf; total time= 19.0s
[CV] END ...C=20, gamma=0.005, kernel=rbf; total time= 37.1s
[CV] END ...C=50, gamma=0.005, kernel=rbf; total time= 36.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 29.5s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 39.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 40.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 52.1s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 45.0s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 16.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 49.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time= 31.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 23.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 37.4s

```

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time= 19.5s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time= 13.6s

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 44.0s

[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 23.4s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 8.6s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 8.8s

[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 5.1s

[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 5.2s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 10.0s

[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time= 6.3s

[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 22.0s

[CV] END ...C=2, gamma=0.001, kernel=rbf; total time= 18.6s

[CV] END ...C=20, gamma=0.0005, kernel=rbf; total time= 17.1s

[CV] END ...C=2, gamma=0.005, kernel=rbf; total time= 36.5s

[CV] END ...C=0.5, gamma=0.0005, kernel=rbf; total time= 21.5s

[CV] END ...C=2, gamma=0.01, kernel=rbf; total time= 40.1s

[CV] END ...C=50, gamma=scale, kernel=rbf; total time= 24.1s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 18.9s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 41.4s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.5min

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.0min

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.5min

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 48.3s

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 26.8s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 48.2s

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 34.3s

[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 13.0s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.4min

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 4.1s

[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time= 2.6s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 6.2s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 23.8s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 23.6s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 10.9s

[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 18.8s

[CV] END ...C=10, gamma=0.0005, kernel=rbf; total time= 16.9s

[CV] END ...C=5, gamma=scale, kernel=rbf; total time= 30.1s

[CV] END ...C=1, gamma=0.0005, kernel=rbf; total time= 21.4s

[CV] END ...C=0.5, gamma=scale, kernel=rbf; total time= 24.6s

[CV] END ...C=1, gamma=scale, kernel=rbf; total time= 24.7s

[CV] END ...C=50, gamma=0.005, kernel=rbf; total time= 36.8s

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 27.7s

[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 39.5s

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 41.3s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,

```

min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 53.6s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 45.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 45.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 25.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time= 36.7s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 34.6s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 31.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 14.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time= 22.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time= 39.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 34.8s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time= 4.9s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time= 4.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time= 12.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 6.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time= 6.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,

```

```

min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 23.3s
[CV] END ...C=10, gamma=0.01, kernel=rbf; total time= 34.9s
[CV] END ...C=5, gamma=0.0005, kernel=rbf; total time= 16.2s
[CV] END ...C=1, gamma=0.0005, kernel=rbf; total time= 19.1s
[CV] END ...C=0.5, gamma=scale, kernel=rbf; total time= 24.0s
[CV] END ...C=2, gamma=0.01, kernel=rbf; total time= 39.5s
[CV] END ...C=0.5, gamma=0.001, kernel=rbf; total time= 20.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 57.1s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 51.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 41.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.7min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 45.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 35.1s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time= 4.7s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time= 4.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time= 12.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 6.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time= 6.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 24.1s
[CV] END ...C=10, gamma=0.01, kernel=rbf; total time= 40.2s
[CV] END ...C=1, gamma=0.001, kernel=rbf; total time= 20.0s
[CV] END ...C=50, gamma=0.001, kernel=rbf; total time= 19.5s

```

[CV] END ...C=20, gamma=0.005, kernel=rbf; total time= 36.6s
 [CV] END ...C=0.5, gamma=0.005, kernel=rbf; total time= 29.5s
 [CV] END ...C=10, gamma=0.001, kernel=rbf; total time= 11.5s
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
 min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
 total time= 17.9s
 [CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
 min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
 subsample=0.7; total time= 43.9s
 [CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
 min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
 subsample=0.9; total time= 2.4min
 [CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
 min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
 subsample=0.9; total time= 2.1min
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
 min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
 total time= 15.4s
 [CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
 min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
 subsample=0.7; total time= 2.7min
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
 min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
 total time= 1.0min
 [CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
 min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
 total time= 45.1s
 [CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
 min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 34.5s
 [CV] END class_weight=balanced, max_depth=8, max_features=log2,
 min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time= 8.2s
 [CV] END class_weight=balanced, max_depth=12, max_features=log2,
 min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time= 5.0s
 [CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
 min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time= 13.2s
 [CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
 min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 11.1s
 [CV] END class_weight=balanced, max_depth=None, max_features=log2,
 min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 9.2s
 [CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
 min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 6.4s
 [CV] END ...C=20, gamma=0.01, kernel=rbf; total time= 35.6s
 [CV] END ...C=2, gamma=0.005, kernel=rbf; total time= 37.7s
 [CV] END ...C=0.5, gamma=0.0005, kernel=rbf; total time= 21.1s
 [CV] END ...C=2, gamma=0.01, kernel=rbf; total time= 39.5s
 [CV] END ...C=0.5, gamma=0.001, kernel=rbf; total time= 20.7s
 [CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
 min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,


```

subsample=0.9; total time= 19.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 44.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 49.6s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 38.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time= 36.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 16.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 32.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 13.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.6min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 5.1s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time= 5.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 21.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 21.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 6.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 10.9s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 19.9s

```

[CV] END ...C=10, gamma=0.0005, kernel=rbf; total time= 14.9s
 [CV] END ...C=5, gamma=scale, kernel=rbf; total time= 21.3s
 [CV] END ...C=2, gamma=0.005, kernel=rbf; total time= 29.1s
 [CV] END ...C=0.5, gamma=scale, kernel=rbf; total time= 16.8s
 [CV] END ...C=20, gamma=0.005, kernel=rbf; total time= 29.5s
 [CV] END ...C=0.5, gamma=0.005, kernel=rbf; total time= 24.5s
 [CV] END ...C=50, gamma=scale, kernel=rbf; total time= 20.5s
 [CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
 min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
 subsample=0.9; total time= 19.5s
 [CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
 min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
 subsample=0.9; total time= 1.1min
 [CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
 min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
 total time= 44.3s
 [CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
 min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
 total time= 1.3min
 [CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
 min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
 subsample=0.9; total time= 2.2min
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
 min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
 total time= 16.2s
 [CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
 min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
 subsample=0.8; total time= 46.9s
 [CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
 min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
 total time= 30.0s
 [CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
 min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
 subsample=0.7; total time= 40.0s
 [CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
 min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
 subsample=0.9; total time= 35.1s
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
 min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
 total time= 59.4s
 [CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
 min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
 total time= 46.8s
 [CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
 min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 36.2s
 [CV] END class_weight=balanced, max_depth=12, max_features=3,
 min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 5.3s
 [CV] END class_weight=balanced, max_depth=12, max_features=3,

```

min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 5.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time= 12.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 11.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 19.3s
[CV] END ...C=10, gamma=0.0005, kernel=rbf; total time= 16.5s
[CV] END ...C=5, gamma=scale, kernel=rbf; total time= 25.6s
[CV] END ...C=5, gamma=0.01, kernel=rbf; total time= 36.9s
[CV] END ...C=20, gamma=0.005, kernel=rbf; total time= 37.4s
[CV] END ...C=0.5, gamma=0.005, kernel=rbf; total time= 30.1s
[CV] END ...C=10, gamma=0.001, kernel=rbf; total time= 10.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 17.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 43.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 49.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 36.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time= 36.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 37.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 45.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 26.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time= 38.7s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 35.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;

```

```

total time= 59.9s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 43.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time= 35.9s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 5.2s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 5.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time= 12.9s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time= 5.4s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 7.8s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 9.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 6.2s
[CV] END ...C=20, gamma=0.01, kernel=rbf; total time= 39.8s
[CV] END ...C=1, gamma=0.001, kernel=rbf; total time= 21.2s
[CV] END ...C=50, gamma=0.001, kernel=rbf; total time= 17.1s
[CV] END ...C=5, gamma=0.005, kernel=rbf; total time= 36.7s
[CV] END ...C=0.5, gamma=0.005, kernel=rbf; total time= 30.8s
[CV] END ...C=10, gamma=0.001, kernel=rbf; total time= 13.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,

```

```

subsample=0.9; total time= 58.6s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 52.5s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 41.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 47.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time= 28.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 21.4s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 37.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 25.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time= 14.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 44.5s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 23.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time= 20.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time= 21.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 22.9s
[CV] END ...C=2, gamma=0.001, kernel=rbf; total time= 19.5s

```

[CV] END ...C=20, gamma=0.0005, kernel=rbf; total time= 13.0s
 [CV] END ...C=5, gamma=0.0005, kernel=rbf; total time= 11.6s
 [CV] END ...C=5, gamma=0.01, kernel=rbf; total time= 31.2s
 [CV] END ...C=5, gamma=0.005, kernel=rbf; total time= 25.8s
 [CV] END ...C=0.5, gamma=0.01, kernel=rbf; total time= 27.7s
 [CV] END ...C=0.5, gamma=0.001, kernel=rbf; total time= 14.3s
 [CV] END ...C=50, gamma=scale, kernel=rbf; total time= 16.1s
 [CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
 min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
 subsample=0.9; total time= 57.0s
 [CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
 min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
 subsample=0.8; total time= 1.1min
 [CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
 min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
 subsample=0.8; total time= 2.2min
 [CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
 min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
 total time= 53.3s
 [CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
 min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
 total time= 39.5s
 [CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
 min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
 subsample=0.7; total time= 2.4min
 [CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
 min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
 total time= 32.3s
 [CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
 min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
 subsample=0.9; total time= 13.3s
 [CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
 min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
 subsample=0.7; total time= 1.4min
 [CV] END class_weight=balanced, max_depth=8, max_features=log2,
 min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 4.0s
 [CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
 min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time= 2.6s
 [CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
 min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 6.2s
 [CV] END class_weight=balanced, max_depth=12, max_features=3,
 min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time= 7.3s
 [CV] END class_weight=balanced, max_depth=12, max_features=3,
 min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time= 7.5s
 [CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
 min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time= 20.6s
 [CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
 min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time= 21.2s

```

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 24.0s
[CV] END ...C=10, gamma=0.01, kernel=rbf; total time= 41.2s
[CV] END ...C=5, gamma=0.01, kernel=rbf; total time= 36.2s
[CV] END ...C=5, gamma=0.005, kernel=rbf; total time= 27.4s
[CV] END ...C=0.5, gamma=0.01, kernel=rbf; total time= 29.1s
[CV] END ...C=0.5, gamma=0.001, kernel=rbf; total time= 14.7s
[CV] END ...C=50, gamma=0.01, kernel=rbf; total time= 11.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 19.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 41.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time= 36.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time= 36.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 36.7s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 45.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 25.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 20.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time= 38.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time= 21.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,

```

```

min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time= 14.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 44.5s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time= 23.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 9.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 9.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 5.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 5.3s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 10.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time= 6.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 24.3s
[CV] END ...C=10, gamma=0.01, kernel=rbf; total time= 40.6s
[CV] END ...C=1, gamma=0.001, kernel=rbf; total time= 19.3s
[CV] END ...C=50, gamma=0.001, kernel=rbf; total time= 18.9s
[CV] END ...C=20, gamma=0.005, kernel=rbf; total time= 37.7s
[CV] END ...C=0.5, gamma=0.005, kernel=rbf; total time= 30.0s
[CV] END ...C=10, gamma=0.001, kernel=rbf; total time= 12.3s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 28.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 39.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 41.9s

```


[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 54.6s

[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 47.5s

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.5min

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 42.8s

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 27.4s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 48.5s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time= 20.1s

[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time= 38.0s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 25.8s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 5.0s

[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time= 5.1s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time= 11.2s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 21.1s

[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 5.0s

[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 5.0s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 9.8s

[CV] END class_weight=balanced, max_depth=16, max_features=3,

```

min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time= 6.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 22.0s
[CV] END ...C=10, gamma=0.0005, kernel=rbf; total time= 17.4s
[CV] END ...C=5, gamma=scale, kernel=rbf; total time= 29.3s
[CV] END ...C=1, gamma=0.0005, kernel=rbf; total time= 20.7s
[CV] END ...C=0.5, gamma=scale, kernel=rbf; total time= 24.6s
[CV] END ...C=1, gamma=scale, kernel=rbf; total time= 25.1s
[CV] END ...C=50, gamma=0.005, kernel=rbf; total time= 37.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 18.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 40.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 15.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 40.2s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time= 31.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 20.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 49.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time= 20.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time= 14.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 44.0s

```

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 4.9s

[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time= 5.1s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time= 11.2s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time= 11.0s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time= 8.3s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time= 8.2s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time= 10.3s

[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time= 5.3s

[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 7.7s

[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 9.1s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time= 0.0s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 6.5s

[CV] END ...C=20, gamma=0.01, kernel=rbf; total time= 34.3s

[CV] END ...C=5, gamma=0.0005, kernel=rbf; total time= 12.2s

[CV] END ...C=5, gamma=0.01, kernel=rbf; total time= 31.7s

[CV] END ...C=5, gamma=0.005, kernel=rbf; total time= 27.5s

[CV] END ...C=0.5, gamma=0.01, kernel=rbf; total time= 29.3s

[CV] END ...C=50, gamma=scale, kernel=rbf; total time= 18.7s

[CV] END ...C=50, gamma=0.01, kernel=rbf; total time= 7.7s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 18.9s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 42.1s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min

[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time= 37.5s

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 48.7s

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 26.2s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 49.0s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 59.4s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time= 5.1s

[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time= 5.1s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 21.1s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 21.9s

[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time= 6.0s

[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time= 5.3s

[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 7.6s

[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 9.2s

[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time= 6.2s

[CV] END ...C=20, gamma=0.01, kernel=rbf; total time= 32.2s

[CV] END ...C=5, gamma=0.0005, kernel=rbf; total time= 14.2s

[CV] END ...C=5, gamma=0.01, kernel=rbf; total time= 40.0s

[CV] END ...C=1, gamma=scale, kernel=rbf; total time= 24.8s

[CV] END ...C=0.5, gamma=0.01, kernel=rbf; total time= 36.7s

[CV] END ...C=50, gamma=0.01, kernel=rbf; total time= 12.7s

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 28.9s

[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 43.2s

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time= 41.5s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 53.5s

[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 46.3s

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.5min

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 46.0s

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 25.9s

[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 47.8s

[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.0min

[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 25.5s

[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time= 15.0s

[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 39.0s

[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 4.1s

[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time= 2.6s

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time= 6.2s

```

[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time= 0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 23.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time= 23.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time= 11.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time= 18.9s
[CV] END ...C=2, gamma=0.001, kernel=rbf; total time= 20.1s
[CV] END ...C=20, gamma=0.0005, kernel=rbf; total time= 16.0s
[CV] END ...C=2, gamma=0.005, kernel=rbf; total time= 36.0s
[CV] END ...C=0.5, gamma=0.0005, kernel=rbf; total time= 22.4s
[CV] END ...C=2, gamma=0.01, kernel=rbf; total time= 39.9s
[CV] END ...C=50, gamma=scale, kernel=rbf; total time= 23.8s

```

PermutationExplainer explainer: 201it [55:03, 16.52s/it]

Shap SHAP: (200, 445)

[SVM] Gráfico salvo com sucesso em 'pipelineAF/svm_importancia_permutacao.png'!

Finalizado fase de treinamento em: 17:45:02

Tempo decorrido: 1:12:16.837437

Fim criação/treino dos modelos

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

```

[18]: '''
      Avaliação e Validação dos modelos
      '''
      print("\n\nIniciando Validação dos modelos\n")
      # verificar importância dos atributos/feature para o Random Forest
      importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
      ↪feature_importances_})

```

```

importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][:-1], top["importance"][:-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
    ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][:-1], top["importance"][:-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
    f"{OUTPUT_DIR}/metricas_comparacao.csv",
    index=False
)

metrics = [

```

```

    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest": rf_model,
    "XGBoost": xgb_model,
    "SVM": svm_model
}

plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[: ,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )

plt.plot([0,1],[0,1], '--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")

```



```
plt.close()

print("Fim avaliação / validação modelos ")
```

Iniciando Validação dos modelos

Top 20 features: Random Forest

	feature	importance
10	AAC_L	0.036460
444	has_mhd	0.033310
433	NUM_NLR_MOTIFS	0.032214
440	dist_glp1_mhd_norm	0.022619
209	DIPEP_LK	0.021724
422	KINASE2	0.020495
204	DIPEP_LE	0.019919
441	has_ploop	0.019235
421	P_LOOP	0.018855
1	AAC_A	0.018749
439	dist_kinase2_glp1_norm	0.017048
250	DIPEP_NL	0.016815
0	protein_length	0.015542
13	AAC_P	0.014321
438	dist_ploop_kinase2_norm	0.013939
6	AAC_G	0.011696
150	DIPEP_HL	0.011192
437	dist_glp1_mhd	0.010893
215	DIPEP_LR	0.010101
190	DIPEP_KL	0.009453

Top 20 features: XGBoost

	feature	importance
444	has_mhd	0.103677
433	NUM_NLR_MOTIFS	0.082959
422	KINASE2	0.031253
10	AAC_L	0.025369
421	P_LOOP	0.024921
441	has_ploop	0.020745
440	dist_glp1_mhd_norm	0.011862
1	AAC_A	0.010955
13	AAC_P	0.008372
209	DIPEP_LK	0.007772
0	protein_length	0.006924
222	DIPEP_MC	0.006822
4	AAC_E	0.005962
442	has_kinase2	0.005844

146	DIPEP_HG	0.005760
333	DIPEP_SP	0.005526
382	DIPEP_WC	0.005506
6	AAC_G	0.005475
150	DIPEP_HL	0.005454
383	DIPEP_WD	0.005276

COMPARAÇÃO MODELOS

	Modelo	ROC_AUC	F1	Precision	Recall	Balanced_Accuracy \
0	Random Forest	0.954422	0.902208	0.916667	0.888199	0.907787
1	XGBoost	0.958014	0.900474	0.916399	0.885093	0.906234
2	SVM	0.952236	0.893293	0.877246	0.909938	0.897706

	Accuracy	MCC
0	0.908824	0.817205
1	0.907353	0.814296
2	0.897059	0.794421

Fim avaliação / validação modelos

```
[19]: '''
      Funções para rodar a aplicação
      '''

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )
```

```

def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file

def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,

```

```

        "protein_seq": protein
    })

candidatos_df = pd.DataFrame(candidatos)
candidatos_df = candidatos_df.sort_values(
    "probabilidade_resistencia",
    ascending=False
)

# top = candidatos_df.head(40)
top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=
↳probabilidade]
print("\nTOP CANDIDATOS com probabilidade de >= ")
print(probabilidade)
print(fasta_nt+", MODELO: "+nmodel)
print(top[[
    "id",
    "descricao",
    "protein_length",
    "probabilidade_resistencia"
]])

validar_motifs(top,planta,nmodel)

candidatos_df.to_csv(
    f"{OUTPUT_DIR}/{output}",
    index=False
)

```

```

[20]: '''
Aplicação - Modelo XGBoost
'''

print("\n\n=====Iniciando Aplicação: Predição de
↳Genes=====\\n\\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\n\\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,
↳xgb_model, 'XGBoost', output="predicoes_arabidopsis_xgboost.
↳csv",planta="Arabidopsis thaliana")

fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

```

=====Iniciando Aplicação: Predição de Genes=====

Predição Modelo XGBoost

Iniciando predição ARABIDOPSIS: 17:45:03

Predição para ARABIDOPSIS

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/arabidopsis_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
4359	NM_001344486.1.p1	NM_001344486.1.p1 GENE.NM_001344486.1~~NM_0013...
4358	NM_001344485.1.p1	NM_001344485.1.p1 GENE.NM_001344485.1~~NM_0013...
4074	NM_001344104.1.p1	NM_001344104.1.p1 GENE.NM_001344104.1~~NM_0013...
8895	NM_122936.5.p1	NM_122936.5.p1 GENE.NM_122936.5~~NM_122936.5.p...
4615	NM_001344786.1.p1	NM_001344786.1.p1 GENE.NM_001344786.1~~NM_0013...
...	...	...
5235	NM_001345597.1.p2	NM_001345597.1.p2 GENE.NM_001345597.1~~NM_0013...
4443	NM_001344582.1.p1	NM_001344582.1.p1 GENE.NM_001344582.1~~NM_0013...
1938	NM_001341343.1.p2	NM_001341343.1.p2 GENE.NM_001341343.1~~NM_0013...
3573	NM_001343468.1.p1	NM_001343468.1.p1 GENE.NM_001343468.1~~NM_0013...
257	NM_001125496.2.p1	NM_001125496.2.p1 GENE.NM_001125496.2~~NM_0011...

	protein_length	probabilidade_resistencia
4359	909	0.999985
4358	909	0.999985
4074	902	0.999977
8895	902	0.999977
4615	909	0.999973
...	...	...
5235	116	0.853222
4443	1213	0.852751
1938	506	0.852373
3573	967	0.851380
257	1896	0.850730

[354 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 226
 KINASE2 : 31
 GLPL : 216
 MHD : 222
 RNBS_A : 0
 RNBS_B : 248
 RNBS_C : 324
 RNBS_D : 319
 HRD : 10
 DFG : 69
 LRR1 : 74
 LRR2 : 145

Finalizado predição ARABIDOPSIS: 17:47:07

Tempo decorrido: 0:02:03.426580

```
[21]: print("\nPredição para MELONGENA\n")
      inicio = datetime.now()
      print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_melongena_fasta,
        ↳xgb_model, 'XGBoost', output="predicoes_melongena_xgboost.csv", planta="Solanum",
        ↳melongena)
      fim = datetime.now()
      print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Predição para MELONGENA

Iniciando predição MELONGENA: 17:47:07

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/melongena_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
3221	GBEF01023314.1.p1	GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010...
3220	GBEF01023313.1.p1	GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010...
9238	GBEF01044458.1.p1	GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010...
9160	GBEF01044249.1.p1	GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010...
9161	GBEF01044250.1.p1	GBEF01044250.1.p1 GENE.GBEF01044250.1~~GBEF010...
...	...	...
4147	GBEF01025373.1.p2	GBEF01025373.1.p2 GENE.GBEF01025373.1~~GBEF010...

```

4149 GBEF01025374.1.p2 GBEF01025374.1.p2 GENE.GBEF01025374.1~~GBEF010...
5262 GBEF01028012.1.p1 GBEF01028012.1.p1 GENE.GBEF01028012.1~~GBEF010...
8409 GBEF01042490.1.p2 GBEF01042490.1.p2 GENE.GBEF01042490.1~~GBEF010...
7239 GBEF01035067.1.p1 GBEF01035067.1.p1 GENE.GBEF01035067.1~~GBEF010...

```

	protein_length	probabilidade_resistencia
3221	872	0.999932
3220	881	0.999834
9238	851	0.999767
9160	579	0.999591
9161	572	0.999448
...	...	...
4147	175	0.863528
4149	175	0.863528
5262	700	0.857689
8409	124	0.855352
7239	411	0.851381

[113 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

```

P_LOOP : 51
KINASE2 : 15
GLPL : 64
MHD : 37
RNBS_A : 0
RNBS_B : 59
RNBS_C : 81
RNBS_D : 100
HRD : 1
DFG : 20
LRR1 : 7
LRR2 : 23

```

Finalizado predição MELONGENA: 17:48:55

Tempo decorrido: 0:01:48.692812

```

[22]: print("\nPredição para TUBEROSUM\n")
      inicio = datetime.now()
      print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_tuberosum_fasta,
        ↳xgb_model, 'XGBoost', output="predicoes_tuberosum_xgboost.csv", planta="Solanum_
        ↳tuberosum")
      fim = datetime.now()

```

```
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Predição para TUBEROSUM

Iniciando predição TUBEROSUM: 17:48:55

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/tuberosum_transcriptoma.fasta, MODELO: XGBoost

	id		descricao \
9242	XM_015312842.1.p1	XM_015312842.1.p1	GENE.XM_015312842.1~~XM_0153...
7062	XM_015303193.1.p1	XM_015303193.1.p1	GENE.XM_015303193.1~~XM_0153...
1731	XM_006355819.2.p1	XM_006355819.2.p1	GENE.XM_006355819.2~~XM_0063...
10090	XM_015314337.1.p1	XM_015314337.1.p1	GENE.XM_015314337.1~~XM_0153...
6583	XM_006367110.2.p1	XM_006367110.2.p1	GENE.XM_006367110.2~~XM_0063...
...	...	...	...
6989	XM_015303028.1.p1	XM_015303028.1.p1	GENE.XM_015303028.1~~XM_0153...
3217	XM_006359200.2.p1	XM_006359200.2.p1	GENE.XM_006359200.2~~XM_0063...
9978	XM_015314104.1.p1	XM_015314104.1.p1	GENE.XM_015314104.1~~XM_0153...
7174	XM_015303449.1.p1	XM_015303449.1.p1	GENE.XM_015303449.1~~XM_0153...
9983	XM_015314108.1.p2	XM_015314108.1.p2	GENE.XM_015314108.1~~XM_0153...

	protein_length	probabilidade_resistencia
9242	1216	0.999998
7062	1057	0.999997
1731	1256	0.999996
10090	840	0.999996
6583	1445	0.999995
...	...	...
6989	535	0.854871
3217	1830	0.854174
9978	1403	0.853747
7174	721	0.852655
9983	133	0.852343

[519 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 331

KINASE2 : 181

GLPL : 371

MHD : 375


```

RNBS_A : 0
RNBS_B : 355
RNBS_C : 451
RNBS_D : 487
HRD : 12
DFG : 80
LRR1 : 97
LRR2 : 187

```

Finalizado predição TUBEROSUM: 17:51:01

Tempo decorrido: 0:02:05.648312

```

[23]: print("\nPredição para PHYSALIS\n")
      inicio = datetime.now()
      print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_physalis_fasta,
        ↳xgb_model, 'XGBoost', output="predicoes_physalis_xgboost.csv", planta="Physalis_
        ↳peruviana")
      fim = datetime.now()
      print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
      print("\nArquivo salvo:")
      print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")

```

Predição para PHYSALIS

Iniciando predição PHYSALIS: 17:51:01

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/physalis_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
9609	IABH01022911.1.p1	IABH01022911.1.p1 GENE.IABH01022911.1~~IABH010...
9618	IABH01022921.1.p1	IABH01022921.1.p1 GENE.IABH01022921.1~~IABH010...
9617	IABH01022919.1.p1	IABH01022919.1.p1 GENE.IABH01022919.1~~IABH010...
10240	IABH01023909.1.p1	IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010...
4194	IABH01013451.1.p1	IABH01013451.1.p1 GENE.IABH01013451.1~~IABH010...
...	...	...
1208	IABH01004575.1.p1	IABH01004575.1.p1 GENE.IABH01004575.1~~IABH010...
2800	IABH01009865.1.p1	IABH01009865.1.p1 GENE.IABH01009865.1~~IABH010...
5488	IABH01015881.1.p1	IABH01015881.1.p1 GENE.IABH01015881.1~~IABH010...
9207	IABH01022198.1.p2	IABH01022198.1.p2 GENE.IABH01022198.1~~IABH010...

2056 IABH01007926.1.p2 IABH01007926.1.p2 GENE.IABH01007926.1~~IABH010...

	protein_length	probabilidade_resistencia
9609	879	0.999992
9618	873	0.999989
9617	881	0.999986
10240	849	0.999986
4194	875	0.999985
...	...	...
1208	838	0.857040
2800	477	0.855058
5488	475	0.854951
9207	117	0.854005
2056	127	0.850058

[201 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 88
KINASE2 : 31
GLPL : 118
MHD : 115
RNBS_A : 0
RNBS_B : 128
RNBS_C : 149
RNBS_D : 177
HRD : 3
DFG : 13
LRR1 : 10
LRR2 : 51

Finalizado predição TUBEROSUM: 17:53:02

Tempo decorrido: 0:02:01.352807

Arquivo salvo:
pipelineAF/predicoes_xgboost.csv

```
[24]: print(df_validacao)
```

```
[{'P_LOOP': 226, 'KINASE2': 31, 'GLPL': 216, 'MHD': 222, 'RNBS_A': 0, 'RNBS_B': 248, 'RNBS_C': 324, 'RNBS_D': 319, 'HRD': 10, 'DFG': 69, 'LRR1': 74, 'LRR2': 145, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 51, 'KINASE2': 15, 'GLPL': 64, 'MHD': 37, 'RNBS_A': 0, 'RNBS_B': 59, 'RNBS_C': 81, 'RNBS_D': 100, 'HRD': 1, 'DFG': 20, 'LRR1': 7, 'LRR2': 23, 'planta': 'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 331, 'KINASE2': 181, 'GLPL': 371,
```

```
'MHD': 375, 'RNBS_A': 0, 'RNBS_B': 355, 'RNBS_C': 451, 'RNBS_D': 487, 'HRD': 12,
'DFG': 80, 'LRR1': 97, 'LRR2': 187, 'planta': 'Solanum tuberosum', 'modelo':
'XGBoost'}, {'P_LOOP': 88, 'KINASE2': 31, 'GLPL': 118, 'MHD': 115, 'RNBS_A': 0,
'RNBS_B': 128, 'RNBS_C': 149, 'RNBS_D': 177, 'HRD': 3, 'DFG': 13, 'LRR1': 10,
'LRR2': 51, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}]
```

```
[25]: '''
Aplicação - Modelo Random Forest
'''

print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,
    rf_model, "RF", output="predicoes_arabidopsis_rf.csv", planta="Arabidopsis_
    thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

Predição Modelo RF

Predição para ARABIDOPSIS

Iniciando predição ARABIDOPSIS: 17:53:02

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/arabidopsis_transcriptoma.fasta, MODELO: RF

	id	descricao \
7063	NM_118853.2.p1	NM_118853.2.p1 GENE.NM_118853.2~NM_118853.2.p...
4174	NM_001344241.1.p1	NM_001344241.1.p1 GENE.NM_001344241.1~NM_0013...
6410	NM_117271.3.p1	NM_117271.3.p1 GENE.NM_117271.3~NM_117271.3.p...
4374	NM_001344502.1.p1	NM_001344502.1.p1 GENE.NM_001344502.1~NM_0013...
4375	NM_001344503.1.p1	NM_001344503.1.p1 GENE.NM_001344503.1~NM_0013...
...	...	...
3619	NM_001343529.1.p1	NM_001343529.1.p1 GENE.NM_001343529.1~NM_0013...
9433	NM_124291.2.p1	NM_124291.2.p1 GENE.NM_124291.2~NM_124291.2.p...
778	NM_001339721.1.p1	NM_001339721.1.p1 GENE.NM_001339721.1~NM_0013...
779	NM_001339722.1.p1	NM_001339722.1.p1 GENE.NM_001339722.1~NM_0013...
780	NM_001339723.1.p1	NM_001339723.1.p1 GENE.NM_001339723.1~NM_0013...

protein_length probabilidade_resistencia

7063	986	0.983823
4174	854	0.974849
6410	1220	0.973435
4374	849	0.973294
4375	849	0.973294
...	...	...
3619	1186	0.859623
9433	981	0.851727
778	2131	0.851576
779	2131	0.851576
780	2131	0.851576

[145 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 145
 KINASE2 : 27
 GLPL : 142
 MHD : 129
 RNBS_A : 0
 RNBS_B : 136
 RNBS_C : 144
 RNBS_D : 145
 HRD : 7
 DFG : 38
 LRR1 : 36
 LRR2 : 88

Finalizado predição ARABIDOPSIS: 18:12:07

Tempo decorrido: 0:19:04.878626

```
[26]: print("\nPredição para MELONGENA\n")
      inicio = datetime.now()
      print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_melongena_fasta,
      ↪rf_model, 'RF', output="predicoes_melongena_rf.csv", planta="Solanum melongena")
      fim = datetime.now()
      print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Predição para MELONGENA

Iniciando predição MELONGENA: 18:12:07

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/melongena_transcriptoma.fasta, MODELO: RF

	id	descricao \
3221	GBEF01023314.1.p1	GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010...
3220	GBEF01023313.1.p1	GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010...
9238	GBEF01044458.1.p1	GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010...
4489	GBEF01026077.1.p1	GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010...
3222	GBEF01023315.1.p1	GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010...
3223	GBEF01023316.1.p1	GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010...
4337	GBEF01025817.1.p1	GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010...
4338	GBEF01025818.1.p1	GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010...
4488	GBEF01026076.1.p1	GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010...
274	GBEF01001042.1.p1	GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010...
9237	GBEF01044457.1.p1	GBEF01044457.1.p1 GENE.GBEF01044457.1~~GBEF010...

	protein_length	probabilidade_resistencia
3221	872	0.967079
3220	881	0.958529
9238	851	0.948912
4489	866	0.938071
3222	883	0.933440
3223	874	0.930708
4337	886	0.930040
4338	886	0.930040
4488	892	0.926730
274	867	0.919010
9237	585	0.865870

VALIDAÇÃO BIOLÓGICA

P_LOOP : 11
KINASE2 : 11
GLPL : 11
MHD : 7
RNBS_A : 0
RNBS_B : 3
RNBS_C : 7
RNBS_D : 11
HRD : 0
DFG : 0
LRR1 : 4
LRR2 : 7

Finalizado predição MELONGENA: 18:28:58

Tempo decorrido: 0:16:50.400590

```
[27]: print("\nPredição para TUBEROSUM\n")
      inicio = datetime.now()
      print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_tuberosum_fasta,
      ↪rf_model, "RF", output="predicoes_tuberosum_rf.csv", planta="Solanum tuberosum")
      fim = datetime.now()
      print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Predição para TUBEROSUM

Iniciando predição TUBEROSUM: 18:28:58

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/tuberosum_transcriptoma.fasta, MODELO: RF

	id	descricao \
1986	XM_006356384.2.p1	XM_006356384.2.p1 GENE.XM_006356384.2~~XM_0063...
7739	XM_015304642.1.p1	XM_015304642.1.p1 GENE.XM_015304642.1~~XM_0153...
7737	XM_015304640.1.p1	XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153...
7738	XM_015304641.1.p1	XM_015304641.1.p1 GENE.XM_015304641.1~~XM_0153...
6718	XM_006367503.2.p1	XM_006367503.2.p1 GENE.XM_006367503.2~~XM_0063...
...	...	...
533	XM_006353220.2.p1	XM_006353220.2.p1 GENE.XM_006353220.2~~XM_0063...
2023	XM_006356481.2.p1	XM_006356481.2.p1 GENE.XM_006356481.2~~XM_0063...
6122	XM_006366010.2.p1	XM_006366010.2.p1 GENE.XM_006366010.2~~XM_0063...
9278	XM_015312959.1.p2	XM_015312959.1.p2 GENE.XM_015312959.1~~XM_0153...
9276	XM_015312958.1.p2	XM_015312958.1.p2 GENE.XM_015312958.1~~XM_0153...

	protein_length	probabilidade_resistencia
1986	1646	0.999961
7739	1267	0.999935
7737	1271	0.999716
7738	1271	0.999707
6718	1242	0.999078
...	...	...
533	907	0.858547
2023	1130	0.854470
6122	523	0.853381
9278	301	0.852637

9276 301 0.852637

[258 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 250
KINASE2 : 161
GLPL : 241
MHD : 240
RNBS_A : 0
RNBS_B : 218
RNBS_C : 245
RNBS_D : 257
HRD : 2
DFG : 52
LRR1 : 77
LRR2 : 124

Finalizado predição TUBEROSUM: 18:48:05

Tempo decorrido: 0:19:07.210802

```
[28]: print("\nPredição para PHYSALIS\n")
      inicio = datetime.now()
      print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_physalis_fasta,
        rf_model, "RF", output="predicoes_physalis_rf.csv", planta="Physalis peruviana")
      fim = datetime.now()
      print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)

      print("\nArquivo salvo:")
      print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

Predição para PHYSALIS

Iniciando predição PHYSALIS: 18:48:05

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/physalis_transcriptoma.fasta, MODELO: RF

id

descricao \

10240	IABH01023909.1.p1	IABH01023909.1.p1	GENE.IABH01023909.1~~IABH010...
10241	IABH01023911.1.p1	IABH01023911.1.p1	GENE.IABH01023911.1~~IABH010...
10242	IABH01023913.1.p1	IABH01023913.1.p1	GENE.IABH01023913.1~~IABH010...
710	IABH01002504.1.p1	IABH01002504.1.p1	GENE.IABH01002504.1~~IABH010...
711	IABH01002505.1.p1	IABH01002505.1.p1	GENE.IABH01002505.1~~IABH010...
10938	IABH01029912.1.p1	IABH01029912.1.p1	GENE.IABH01029912.1~~IABH010...
4194	IABH01013451.1.p1	IABH01013451.1.p1	GENE.IABH01013451.1~~IABH010...
9609	IABH01022911.1.p1	IABH01022911.1.p1	GENE.IABH01022911.1~~IABH010...
10218	IABH01023856.1.p1	IABH01023856.1.p1	GENE.IABH01023856.1~~IABH010...
9617	IABH01022919.1.p1	IABH01022919.1.p1	GENE.IABH01022919.1~~IABH010...
7797	IABH01019816.1.p1	IABH01019816.1.p1	GENE.IABH01019816.1~~IABH010...
1483	IABH01005634.1.p1	IABH01005634.1.p1	GENE.IABH01005634.1~~IABH010...
10219	IABH01023859.1.p1	IABH01023859.1.p1	GENE.IABH01023859.1~~IABH010...
10220	IABH01023861.1.p1	IABH01023861.1.p1	GENE.IABH01023861.1~~IABH010...
9265	IABH01022284.1.p1	IABH01022284.1.p1	GENE.IABH01022284.1~~IABH010...
1820	IABH01007074.1.p1	IABH01007074.1.p1	GENE.IABH01007074.1~~IABH010...
9618	IABH01022921.1.p1	IABH01022921.1.p1	GENE.IABH01022921.1~~IABH010...
3828	IABH01012351.1.p1	IABH01012351.1.p1	GENE.IABH01012351.1~~IABH010...
10217	IABH01023853.1.p1	IABH01023853.1.p1	GENE.IABH01023853.1~~IABH010...
4477	IABH01013953.1.p1	IABH01013953.1.p1	GENE.IABH01013953.1~~IABH010...
4478	IABH01013954.1.p1	IABH01013954.1.p1	GENE.IABH01013954.1~~IABH010...
10249	IABH01023929.1.p1	IABH01023929.1.p1	GENE.IABH01023929.1~~IABH010...
39	IABH01000147.1.p1	IABH01000147.1.p1	GENE.IABH01000147.1~~IABH010...
10950	IABH01030004.1.p1	IABH01030004.1.p1	GENE.IABH01030004.1~~IABH010...
9434	IABH01022586.1.p1	IABH01022586.1.p1	GENE.IABH01022586.1~~IABH010...
200	IABH01000745.1.p1	IABH01000745.1.p1	GENE.IABH01000745.1~~IABH010...
201	IABH01000746.1.p1	IABH01000746.1.p1	GENE.IABH01000746.1~~IABH010...
724	IABH01002537.1.p1	IABH01002537.1.p1	GENE.IABH01002537.1~~IABH010...
725	IABH01002538.1.p1	IABH01002538.1.p1	GENE.IABH01002538.1~~IABH010...
9482	IABH01022656.1.p1	IABH01022656.1.p1	GENE.IABH01022656.1~~IABH010...
9479	IABH01022654.1.p1	IABH01022654.1.p1	GENE.IABH01022654.1~~IABH010...
2687	IABH01009585.1.p1	IABH01009585.1.p1	GENE.IABH01009585.1~~IABH010...
5100	IABH01015140.1.p1	IABH01015140.1.p1	GENE.IABH01015140.1~~IABH010...
6495	IABH01017695.1.p1	IABH01017695.1.p1	GENE.IABH01017695.1~~IABH010...
9481	IABH01022655.1.p1	IABH01022655.1.p1	GENE.IABH01022655.1~~IABH010...
679	IABH01002433.1.p1	IABH01002433.1.p1	GENE.IABH01002433.1~~IABH010...
7151	IABH01018818.1.p1	IABH01018818.1.p1	GENE.IABH01018818.1~~IABH010...
4957	IABH01014861.1.p1	IABH01014861.1.p1	GENE.IABH01014861.1~~IABH010...
4958	IABH01014862.1.p1	IABH01014862.1.p1	GENE.IABH01014862.1~~IABH010...
9950	IABH01023452.1.p1	IABH01023452.1.p1	GENE.IABH01023452.1~~IABH010...
9068	IABH01021963.1.p1	IABH01021963.1.p1	GENE.IABH01021963.1~~IABH010...
9067	IABH01021962.1.p1	IABH01021962.1.p1	GENE.IABH01021962.1~~IABH010...
9069	IABH01021964.1.p1	IABH01021964.1.p1	GENE.IABH01021964.1~~IABH010...
6822	IABH01018289.1.p1	IABH01018289.1.p1	GENE.IABH01018289.1~~IABH010...
1171	IABH01004520.1.p1	IABH01004520.1.p1	GENE.IABH01004520.1~~IABH010...

	protein_length	probabilidade_resistencia
10240	849	0.993898

10241	852	0.989405
10242	852	0.989405
710	889	0.988133
711	889	0.988133
10938	890	0.987008
4194	875	0.986780
9609	879	0.984730
10218	866	0.982854
9617	881	0.982730
7797	881	0.981809
1483	908	0.980163
10219	870	0.979756
10220	865	0.979108
9265	960	0.975559
1820	892	0.974636
9618	873	0.972635
3828	952	0.970606
10217	726	0.969484
4477	864	0.960720
4478	864	0.960720
10249	530	0.960000
39	735	0.956535
10950	888	0.952276
9434	903	0.947768
200	891	0.940137
201	891	0.940137
724	1395	0.937856
725	1286	0.936183
9482	1390	0.927557
9479	1343	0.925579
2687	1247	0.913702
5100	845	0.911040
6495	593	0.900655
9481	715	0.897377
679	1174	0.892376
7151	1208	0.884074
4957	884	0.882841
4958	884	0.882841
9950	423	0.875856
9068	962	0.875160
9067	962	0.875160
9069	962	0.875160
6822	762	0.873132
1171	1326	0.858656

VALIDAÇÃO BIOLÓGICA

P_LOOP : 43

KINASE2 : 27
GLPL : 40
MHD : 44
RNBS_A : 0
RNBS_B : 42
RNBS_C : 44
RNBS_D : 45
HRD : 0
DFG : 5
LRR1 : 6
LRR2 : 26

Finalizado predição PHYSALIS: 19:08:33

Tempo decorrido: 0:20:27.700196

Arquivo salvo:
pipelineAF/predicoes_rf.csv

```
[29]: '''  
      Aplicação - Modelo SVM  
      '''  
      print("\n\n Predição Modelo SVM")  
      print("\nPredição para ARABIDOPSIS\n")  
      inicio = datetime.now()  
      print("\n\nIniciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))  
      predizer_transcritos(transcriptoma_arabidopsis_fasta,▯  
      ↪svm_model, "SVM", output="predicoes_arabidopsis_svm.csv", planta="Arabidopsis▯  
      ↪thaliana")  
      fim = datetime.now()  
      print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))  
      print("\nTempo decorrido:", fim - inicio)
```

Predição Modelo SVM

Predição para ARABIDOPSIS

Iniciando predição ARABIDOPSIS: 19:08:33

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAF/arabidopsis_transcriptoma.fasta, MODELO: SVM

	id	descricao \
4373	NM_001344501.1.p1	NM_001344501.1.p1 GENE.NM_001344501.1~~NM_0013...
4376	NM_001344504.1.p1	NM_001344504.1.p1 GENE.NM_001344504.1~~NM_0013...
4375	NM_001344503.1.p1	NM_001344503.1.p1 GENE.NM_001344503.1~~NM_0013...
4374	NM_001344502.1.p1	NM_001344502.1.p1 GENE.NM_001344502.1~~NM_0013...
9182	NM_123739.3.p1	NM_123739.3.p1 GENE.NM_123739.3~~NM_123739.3.p...
...	...	...
1506	NM_001340764.1.p1	NM_001340764.1.p1 GENE.NM_001340764.1~~NM_0013...
1509	NM_001340766.1.p1	NM_001340766.1.p1 GENE.NM_001340766.1~~NM_0013...
2961	NM_001342680.1.p1	NM_001342680.1.p1 GENE.NM_001342680.1~~NM_0013...
8466	NM_121795.4.p1	NM_121795.4.p1 GENE.NM_121795.4~~NM_121795.4.p...
4029	NM_001344037.1.p1	NM_001344037.1.p1 GENE.NM_001344037.1~~NM_0013...

	protein_length	probabilidade_resistencia
4373	849	0.997195
4376	849	0.997195
4375	849	0.997195
4374	849	0.997195
9182	849	0.997195
...	...	...
1506	1769	0.854080
1509	1769	0.854080
2961	837	0.853793
8466	1614	0.853449
4029	373	0.850351

[249 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 182
 KINASE2 : 27
 GLPL : 171
 MHD : 163
 RNBS_A : 0
 RNBS_B : 188
 RNBS_C : 231
 RNBS_D : 238
 HRD : 7
 DFG : 55
 LRR1 : 44
 LRR2 : 111

Finalizado predição ARABIDOPSIS: 19:08:37

Tempo decorrido: 0:00:04.904387

```
[30]: print("\nPredição para MELONGENA\n")
      inicio = datetime.now()
      print("\n\nIniciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_melongena_fasta,
        ↪svm_model, 'SVM', output="predicoes_melongena_svm.csv", planta="Solanum_
        ↪melongena")
      fim = datetime.now()
      print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
```

Predição para MELONGENA

Iniciando predição MELONGENA: 19:08:37

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/melongena_transcriptoma.fasta, MODELO: SVM

	id	descricao \
3221	GBEF01023314.1.p1	GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010...
4488	GBEF01026076.1.p1	GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010...
4489	GBEF01026077.1.p1	GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010...
3223	GBEF01023316.1.p1	GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010...
3220	GBEF01023313.1.p1	GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010...
9160	GBEF01044249.1.p1	GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010...
3222	GBEF01023315.1.p1	GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010...
9161	GBEF01044250.1.p1	GBEF01044250.1.p1 GENE.GBEF01044250.1~~GBEF010...
789	GBEF01005143.1.p1	GBEF01005143.1.p1 GENE.GBEF01005143.1~~GBEF010...
274	GBEF01001042.1.p1	GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010...
9162	GBEF01044251.1.p1	GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010...
9163	GBEF01044252.1.p1	GBEF01044252.1.p1 GENE.GBEF01044252.1~~GBEF010...
9158	GBEF01044247.1.p1	GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010...
4337	GBEF01025817.1.p1	GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010...
4338	GBEF01025818.1.p1	GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010...
9159	GBEF01044248.1.p1	GBEF01044248.1.p1 GENE.GBEF01044248.1~~GBEF010...
2372	GBEF01017801.1.p1	GBEF01017801.1.p1 GENE.GBEF01017801.1~~GBEF010...
9238	GBEF01044458.1.p1	GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010...
631	GBEF01002344.1.p1	GBEF01002344.1.p1 GENE.GBEF01002344.1~~GBEF010...
1067	GBEF01015093.1.p1	GBEF01015093.1.p1 GENE.GBEF01015093.1~~GBEF010...
7823	GBEF01041323.1.p1	GBEF01041323.1.p1 GENE.GBEF01041323.1~~GBEF010...
7023	GBEF01034428.1.p1	GBEF01034428.1.p1 GENE.GBEF01034428.1~~GBEF010...
1608	GBEF01016312.1.p1	GBEF01016312.1.p1 GENE.GBEF01016312.1~~GBEF010...
4586	GBEF01026239.1.p2	GBEF01026239.1.p2 GENE.GBEF01026239.1~~GBEF010...
7022	GBEF01034427.1.p1	GBEF01034427.1.p1 GENE.GBEF01034427.1~~GBEF010...
9057	GBEF01044016.1.p1	GBEF01044016.1.p1 GENE.GBEF01044016.1~~GBEF010...

5471	GBEF01028519.1.p1	GBEF01028519.1.p1	GENE.GBEF01028519.1~~GBEF010...
5478	GBEF01028524.1.p1	GBEF01028524.1.p1	GENE.GBEF01028524.1~~GBEF010...
5475	GBEF01028522.1.p1	GBEF01028522.1.p1	GENE.GBEF01028522.1~~GBEF010...
5472	GBEF01028520.1.p1	GBEF01028520.1.p1	GENE.GBEF01028520.1~~GBEF010...
5488	GBEF01028531.1.p1	GBEF01028531.1.p1	GENE.GBEF01028531.1~~GBEF010...
5482	GBEF01028527.1.p1	GBEF01028527.1.p1	GENE.GBEF01028527.1~~GBEF010...
5484	GBEF01028528.1.p1	GBEF01028528.1.p1	GENE.GBEF01028528.1~~GBEF010...
5995	GBEF01030153.1.p1	GBEF01030153.1.p1	GENE.GBEF01030153.1~~GBEF010...
632	GBEF01002345.1.p1	GBEF01002345.1.p1	GENE.GBEF01002345.1~~GBEF010...
6005	GBEF01030217.1.p1	GBEF01030217.1.p1	GENE.GBEF01030217.1~~GBEF010...
5489	GBEF01028532.1.p1	GBEF01028532.1.p1	GENE.GBEF01028532.1~~GBEF010...
8624	GBEF01043062.1.p1	GBEF01043062.1.p1	GENE.GBEF01043062.1~~GBEF010...
4918	GBEF01027126.1.p1	GBEF01027126.1.p1	GENE.GBEF01027126.1~~GBEF010...
4917	GBEF01027125.1.p1	GBEF01027125.1.p1	GENE.GBEF01027125.1~~GBEF010...
5287	GBEF01028088.1.p1	GBEF01028088.1.p1	GENE.GBEF01028088.1~~GBEF010...
4919	GBEF01027127.1.p1	GBEF01027127.1.p1	GENE.GBEF01027127.1~~GBEF010...
9237	GBEF01044457.1.p1	GBEF01044457.1.p1	GENE.GBEF01044457.1~~GBEF010...
9241	GBEF01044460.1.p1	GBEF01044460.1.p1	GENE.GBEF01044460.1~~GBEF010...
3583	GBEF01024178.1.p2	GBEF01024178.1.p2	GENE.GBEF01024178.1~~GBEF010...
5285	GBEF01028087.1.p1	GBEF01028087.1.p1	GENE.GBEF01028087.1~~GBEF010...
8623	GBEF01043061.1.p1	GBEF01043061.1.p1	GENE.GBEF01043061.1~~GBEF010...
8978	GBEF01043843.1.p1	GBEF01043843.1.p1	GENE.GBEF01043843.1~~GBEF010...
1303	GBEF01015662.1.p1	GBEF01015662.1.p1	GENE.GBEF01015662.1~~GBEF010...
6260	GBEF01031768.1.p1	GBEF01031768.1.p1	GENE.GBEF01031768.1~~GBEF010...
6998	GBEF01034339.1.p2	GBEF01034339.1.p2	GENE.GBEF01034339.1~~GBEF010...
6999	GBEF01034340.1.p2	GBEF01034340.1.p2	GENE.GBEF01034340.1~~GBEF010...
7001	GBEF01034341.1.p1	GBEF01034341.1.p1	GENE.GBEF01034341.1~~GBEF010...
2099	GBEF01017282.1.p3	GBEF01017282.1.p3	GENE.GBEF01017282.1~~GBEF010...
2102	GBEF01017285.1.p3	GBEF01017285.1.p3	GENE.GBEF01017285.1~~GBEF010...
9250	GBEF01044472.1.p1	GBEF01044472.1.p1	GENE.GBEF01044472.1~~GBEF010...

	protein_length	probabilidade_resistencia
3221	872	0.999993
4488	892	0.999992
4489	866	0.999986
3223	874	0.999981
3220	881	0.997285
9160	579	0.996130
3222	883	0.995631
9161	572	0.993401
789	504	0.992675
274	867	0.992602
9162	532	0.992209
9163	525	0.986125
9158	713	0.985967
4337	886	0.980215
4338	886	0.980215
9159	706	0.978194

2372	611	0.975541
9238	851	0.973596
631	359	0.973024
1067	1025	0.960725
7823	311	0.953419
7023	1126	0.950860
1608	865	0.946610
4586	408	0.946272
7022	1128	0.940296
9057	711	0.939846
5471	500	0.925280
5478	500	0.925280
5475	500	0.925280
5472	500	0.925280
5488	500	0.925280
5482	500	0.925280
5484	500	0.925280
5995	430	0.924219
632	311	0.919771
6005	532	0.909949
5489	548	0.905963
8624	482	0.902553
4918	420	0.901129
4917	420	0.901129
5287	486	0.895095
4919	423	0.887959
9237	585	0.886695
9241	454	0.886206
3583	174	0.879498
5285	293	0.872373
8623	468	0.870951
8978	551	0.866501
1303	395	0.861803
6260	528	0.860107
6998	121	0.859503
6999	121	0.859503
7001	121	0.859503
2099	118	0.857759
2102	118	0.857759
9250	563	0.851785

VALIDAÇÃO BIOLÓGICA

P_LOOP : 31
 KINASE2 : 13
 GLPL : 40
 MHD : 16
 RNBS_A : 0

RNBS_B : 31
 RNBS_C : 40
 RNBS_D : 50
 HRD : 1
 DFG : 9
 LRR1 : 4
 LRR2 : 10

Finalizado predição MELONGENA: 19:08:42

Tempo decorrido: 0:00:04.120297

```

[31]: print("\nPredição para TUBEROSUM\n")
      inicio = datetime.now()
      print("\n\nIniciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_tuberosum_fasta,
        ↪svm_model, "SVM", output="predicoes_tuberosum_svm.csv", planta="Solanum_
        ↪tuberosum")
      fim = datetime.now()
      print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)
  
```

Predição para TUBEROSUM

Iniciando predição TUBEROSUM: 19:08:42

TOP CANDIDATOS com probabilidade de >=
 0.85

pipelineAF/tuberosum_transcriptoma.fasta, MODELO: SVM

	id	descricao \
9242	XM_015312842.1.p1	XM_015312842.1.p1 GENE.XM_015312842.1~~XM_0153...
6820	XM_006367868.2.p1	XM_006367868.2.p1 GENE.XM_006367868.2~~XM_0063...
1960	XM_006356330.2.p1	XM_006356330.2.p1 GENE.XM_006356330.2~~XM_0063...
6821	XM_006367869.2.p1	XM_006367869.2.p1 GENE.XM_006367869.2~~XM_0063...
7737	XM_015304640.1.p1	XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153...
...	...	...
3027	XM_006358737.2.p1	XM_006358737.2.p1 GENE.XM_006358737.2~~XM_0063...
3025	XM_006358734.2.p1	XM_006358734.2.p1 GENE.XM_006358734.2~~XM_0063...
8752	XM_015311808.1.p1	XM_015311808.1.p1 GENE.XM_015311808.1~~XM_0153...
7761	XM_015304671.1.p1	XM_015304671.1.p1 GENE.XM_015304671.1~~XM_0153...
1665	XM_006355651.2.p1	XM_006355651.2.p1 GENE.XM_006355651.2~~XM_0063...

protein_length probabilidade_resistencia

9242	1216	1.000000
6820	933	1.000000
1960	1279	1.000000
6821	933	1.000000
7737	1271	1.000000
...	...	...
3027	462	0.856692
3025	462	0.856692
8752	1375	0.856278
7761	227	0.853727
1665	513	0.851223

[405 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 302
 KINASE2 : 173
 GLPL : 323
 MHD : 324
 RNBS_A : 0
 RNBS_B : 296
 RNBS_C : 364
 RNBS_D : 390
 HRD : 4
 DFG : 59
 LRR1 : 91
 LRR2 : 167

Finalizado predição TUBEROSUM: 19:08:46

Tempo decorrido: 0:00:04.911159

```
[32]: print("\nPredição para PHYSALIS\n")
      inicio = datetime.now()
      print("\n\nIniciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
      predizer_transcritos(transcriptoma_physalis_fasta,
        ↪svm_model,"SVM",output="predicoes_physalis_svm.csv",planta="Physalis_
        ↪peruviana")
      fim = datetime.now()
      print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
      print("\nTempo decorrido:", fim - inicio)

      print("\nArquivo salvo:")
      print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```


Predição para PHYSALIS

Iniciando predição PHYSALIS: 19:08:46

TOP CANDIDATOS com probabilidade de >= 0.85

pipelineAF/physalis_transcriptoma.fasta, MODELO: SVM

	id		descricao \
10249	IABH01023929.1.p1	IABH01023929.1.p1	GENE.IABH01023929.1~~IABH010...
9950	IABH01023452.1.p1	IABH01023452.1.p1	GENE.IABH01023452.1~~IABH010...
10240	IABH01023909.1.p1	IABH01023909.1.p1	GENE.IABH01023909.1~~IABH010...
9618	IABH01022921.1.p1	IABH01022921.1.p1	GENE.IABH01022921.1~~IABH010...
9609	IABH01022911.1.p1	IABH01022911.1.p1	GENE.IABH01022911.1~~IABH010...
...	...	...	...
8004	IABH01020145.1.p2	IABH01020145.1.p2	GENE.IABH01020145.1~~IABH010...
5018	IABH01014981.1.p1	IABH01014981.1.p1	GENE.IABH01014981.1~~IABH010...
5017	IABH01014980.1.p1	IABH01014980.1.p1	GENE.IABH01014980.1~~IABH010...
6313	IABH01017323.1.p1	IABH01017323.1.p1	GENE.IABH01017323.1~~IABH010...
1170	IABH01004519.1.p1	IABH01004519.1.p1	GENE.IABH01004519.1~~IABH010...

	protein_length	probabilidade_resistencia
10249	530	1.000000
9950	423	1.000000
10240	849	0.999999
9618	873	0.999999
9609	879	0.999999
...	...	...
8004	133	0.865972
5018	677	0.862121
5017	677	0.862121
6313	376	0.861819
1170	1048	0.860780

[115 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 61
 KINASE2 : 30
 GLPL : 78
 MHD : 77
 RNBS_A : 0
 RNBS_B : 83
 RNBS_C : 100
 RNBS_D : 105

HRD : 2
DFG : 11
LRR1 : 8
LRR2 : 36

Finalizado predição PHYSALIS: 19:08:51

Tempo decorrido: 0:00:05.018272

Arquivo salvo:
pipelineAF/predicoes_svm.csv

```
[33]: print(df_validacao)

[{'P_LOOP': 226, 'KINASE2': 31, 'GLPL': 216, 'MHD': 222, 'RNBS_A': 0, 'RNBS_B': 248, 'RNBS_C': 324, 'RNBS_D': 319, 'HRD': 10, 'DFG': 69, 'LRR1': 74, 'LRR2': 145, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 51, 'KINASE2': 15, 'GLPL': 64, 'MHD': 37, 'RNBS_A': 0, 'RNBS_B': 59, 'RNBS_C': 81, 'RNBS_D': 100, 'HRD': 1, 'DFG': 20, 'LRR1': 7, 'LRR2': 23, 'planta': 'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 331, 'KINASE2': 181, 'GLPL': 371, 'MHD': 375, 'RNBS_A': 0, 'RNBS_B': 355, 'RNBS_C': 451, 'RNBS_D': 487, 'HRD': 12, 'DFG': 80, 'LRR1': 97, 'LRR2': 187, 'planta': 'Solanum tuberosum', 'modelo': 'XGBoost'}, {'P_LOOP': 88, 'KINASE2': 31, 'GLPL': 118, 'MHD': 115, 'RNBS_A': 0, 'RNBS_B': 128, 'RNBS_C': 149, 'RNBS_D': 177, 'HRD': 3, 'DFG': 13, 'LRR1': 10, 'LRR2': 51, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}, {'P_LOOP': 145, 'KINASE2': 27, 'GLPL': 142, 'MHD': 129, 'RNBS_A': 0, 'RNBS_B': 136, 'RNBS_C': 144, 'RNBS_D': 145, 'HRD': 7, 'DFG': 38, 'LRR1': 36, 'LRR2': 88, 'planta': 'Arabidopsis thaliana', 'modelo': 'RF'}, {'P_LOOP': 11, 'KINASE2': 11, 'GLPL': 11, 'MHD': 7, 'RNBS_A': 0, 'RNBS_B': 3, 'RNBS_C': 7, 'RNBS_D': 11, 'HRD': 0, 'DFG': 0, 'LRR1': 4, 'LRR2': 7, 'planta': 'Solanum melongena', 'modelo': 'RF'}, {'P_LOOP': 250, 'KINASE2': 161, 'GLPL': 241, 'MHD': 240, 'RNBS_A': 0, 'RNBS_B': 218, 'RNBS_C': 245, 'RNBS_D': 257, 'HRD': 2, 'DFG': 52, 'LRR1': 77, 'LRR2': 124, 'planta': 'Solanum tuberosum', 'modelo': 'RF'}, {'P_LOOP': 43, 'KINASE2': 27, 'GLPL': 40, 'MHD': 44, 'RNBS_A': 0, 'RNBS_B': 42, 'RNBS_C': 44, 'RNBS_D': 45, 'HRD': 0, 'DFG': 5, 'LRR1': 6, 'LRR2': 26, 'planta': 'Physalis peruviana', 'modelo': 'RF'}, {'P_LOOP': 182, 'KINASE2': 27, 'GLPL': 171, 'MHD': 163, 'RNBS_A': 0, 'RNBS_B': 188, 'RNBS_C': 231, 'RNBS_D': 238, 'HRD': 7, 'DFG': 55, 'LRR1': 44, 'LRR2': 111, 'planta': 'Arabidopsis thaliana', 'modelo': 'SVM'}, {'P_LOOP': 31, 'KINASE2': 13, 'GLPL': 40, 'MHD': 16, 'RNBS_A': 0, 'RNBS_B': 31, 'RNBS_C': 40, 'RNBS_D': 50, 'HRD': 1, 'DFG': 9, 'LRR1': 4, 'LRR2': 10, 'planta': 'Solanum melongena', 'modelo': 'SVM'}, {'P_LOOP': 302, 'KINASE2': 173, 'GLPL': 323, 'MHD': 324, 'RNBS_A': 0, 'RNBS_B': 296, 'RNBS_C': 364, 'RNBS_D': 390, 'HRD': 4, 'DFG': 59, 'LRR1': 91, 'LRR2': 167, 'planta': 'Solanum tuberosum', 'modelo': 'SVM'}, {'P_LOOP': 61, 'KINASE2': 30, 'GLPL': 78, 'MHD': 77, 'RNBS_A': 0, 'RNBS_B': 83, 'RNBS_C': 100, 'RNBS_D': 105, 'HRD': 2, 'DFG': 11, 'LRR1': 8, 'LRR2': 36, 'planta': 'Physalis peruviana', 'modelo': 'SVM'}]
```

```
[34]: df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

	planta	modelo	motivo	contagem
0	Arabidopsis thaliana	XGBoost	P_LOOP	226
1	Solanum melongena	XGBoost	P_LOOP	51
2	Solanum tuberosum	XGBoost	P_LOOP	331
3	Physalis peruviana	XGBoost	P_LOOP	88
4	Arabidopsis thaliana	RF	P_LOOP	145

```
[35]: #Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
```

```

        size="contagem",
        sizes=(50,600)
    )
    # plt.title("Distribuição dos motivos por modelo e espécie")
    plt.tight_layout()
    plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
    plt.close()
    # plt.show()

#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
    height=5,
    aspect=1.2
)
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,
#           fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)

```

```

plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,
#           fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão

```

```

)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,
↳fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,
↳fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",

```

```

        columns="grupo",
        values="contagem",
        fill_value=0
    )

plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",
#           ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
           ↪png", dpi=300)
plt.close()
# plt.show()

print("===== Fim do Pipeline =====")

```

```
===== Fim do Pipeline =====
```